

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO,
ENGENHARIA ELETRÔNICA E ENGENHARIA CARTOGRÁFICA**

**RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA**

**CAIXÃO DE AREIA COM REALIDADE AUMENTADA: SISTEMA DE
VISUALIZAÇÃO INTERATIVA DE MODELOS DIGITAIS DE ELEVAÇÃO PARA
APOIO AO TREINAMENTO MILITAR**

**RIO DE JANEIRO
2026**

©2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmар ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

de Figueiredo Schuinki, Rafael; Belchior Façanha Nogueira, Raquel; da Silva Maldonado, Mateus; Johnatas Ribeiro Batista, Estêvão.

Caixão de Areia com Realidade Aumentada: Sistema de Visualização Interativa de Modelos Digitais de Elevação para Apoio ao Treinamento Militar / Rafael de Figueiredo Schuinki, Raquel Belchior Façanha Nogueira, Mateus da Silva Maldonado e Estêvão Johnatas Ribeiro Batista. – Rio de Janeiro, 2026. 66 f.

Orientador(es): Daniel Rodrigues dos Santos, Leonardo Assumpção Moreira, Bruno Eduardo Madeira e Gabriel da Cruz Fontenelle.

Projeto de Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica, 2026.

1. realidade aumentada. 2. modelo digital de elevação. 3. kinect. 4. calibração geométrica. 5. RANSAC. 6. treinamento militar. i. Rodrigues dos Santos, Daniel (orient.) ii. Assumpção Moreira, Leonardo (orient.) iii. Eduardo Madeira, Bruno (orient.) iv. da Cruz Fontenelle, Gabriel (orient.) v. Título

RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA

CAIXÃO DE AREIA COM REALIDADE AUMENTADA: SISTEMA DE
VISUALIZAÇÃO INTERATIVA DE MODELOS DIGITAIS DE ELEVAÇÃO
PARA APOIO AO TREINAMENTO MILITAR

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica.

Orientador(es): Daniel Rodrigues dos Santos, D.Sc.
Leonardo Assumpção Moreira, D.Sc.
Bruno Eduardo Madeira, D.Sc.
Gabriel da Cruz Fontenelle, M.Sc.

Rio de Janeiro

2026

**RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA**

**Caixão de Areia com Realidade Aumentada: Sistema de
Visualização Interativa de Modelos Digitais de Elevação
para Apoio ao Treinamento Militar**

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica.

Orientador(es): Daniel Rodrigues dos Santos, Leonardo Assumpção Moreira, Bruno Eduardo Madeira e Gabriel da Cruz Fontenelle.

Aprovada em 08 de outubro de 2026, pela seguinte banca examinadora:

Prof. Daniel Rodrigues dos Santos - D.Sc. do IME - Presidente

Prof. Leonardo Assumpção Moreira - D.Sc. do IME

Prof. Bruno Eduardo Madeira - D.Sc. do IME

Prof. Gabriel da Cruz Fontenelle - M.Sc. do IME

Rio de Janeiro
2026

*Aos nossos familiares, que sempre acreditaram
no poder transformador da educação técnica
e da pesquisa aplicada à defesa nacional.*

AGRADECIMENTOS

Agradecemos ao Instituto Militar de Engenharia (IME) pela formação técnica, científica e humana de excelência, que possibilitou a concepção e o desenvolvimento deste trabalho.

Agradecemos especialmente aos nossos orientadores — Prof. Daniel Rodrigues dos Santos, Prof. Leonardo Assumpção Moreira, Prof. Bruno Eduardo Madeira e Prof. Gabriel da Cruz Fontenelle — pelas inúmeras horas de discussão técnica, pelas sugestões metodológicas e pelo rigor acadêmico transmitidos ao longo de todo o projeto.

À Seção de Simulação da Academia Militar das Agulhas Negras (AMAN), pela demanda real que deu origem a este trabalho e pela disponibilidade em compartilhar sua experiência operacional com caixões de areia físicos.

Aos nossos familiares, pela paciência, pelo apoio incondicional e pela compreensão nos inúmeros fins de semana dedicados a esta obra.

Aos colegas de turma, com quem partilhamos discussões técnicas, código, café e amizade.

Por fim, à comunidade de *software livre* — em especial aos mantenedores do NumPy, OpenCV, Open3D e do template abntex2ime — sem cuja generosidade este trabalho jamais teria sido viável.

*“A guerra é o reino da incerteza;
três quartos das coisas em que se baseia
a ação na guerra estão envoltos
em uma névoa de incerteza maior ou menor.”*
— *Carl von Clausewitz, Da Guerra, 1832*

RESUMO

Este trabalho apresenta o projeto e a implementação de um sistema de Caixaão de Areia com Realidade Aumentada voltado ao treinamento militar em operações no terreno, desenvolvido em atendimento a uma demanda real da Seção de Simulação da Academia Militar das Agulhas Negras (AMAN). O sistema integra um sensor de profundidade Microsoft Kinect v2, um projetor convencional e um Modelo Digital de Elevação (MDE) de referência, fornecendo *feedback* visual em tempo real — **vermelho** para excesso de areia, **azul** para falta e **verde** para conformidade — diretamente sobre a superfície da areia. A calibração geométrica Kinect $\rightarrow$ Mesa é obtida de forma empírica e automática a partir de uma superfície plana de referência (uma tampa apoiada sobre o caixaão ou a própria base vazia): como o campo de visão do sensor é mais largo que a área útil do caixaão, o algoritmo RANSAC isola primeiro o maior conjunto de pontos coplanares, descartando moldura, piso e ruído, e a normal do plano é então refinada por decomposição em valores singulares (SVD) apenas sobre esses *inliers*; na sequência, constroem-se bases ortonormais por Gram-Schmidt e transformações afins homogêneas que levam a cota zero à base de madeira do caixaão. A projeção Mesa $\rightarrow$ Projetor emprega o modelo *pinhole* de Tsai. A renderização emprega malha discretizada de 30×30 células com agregação espacial para filtragem estatística do ruído do sensor. A arquitetura monolítica modularizada dispõe de cadeia de *fallback* em quatro níveis (PyKinect2, Open3D, libfreenect2 e simulação interativa via *mouse*), assegurando operação mesmo na ausência de hardware. A qualidade do motor matemático é assegurada por uma suíte de 83 testes unitários automatizados, todos aprovados, cobrindo todos os contratos algébricos do sistema. O sistema foi validado tanto em modo simulação quanto em ensaios de campo com o sensor Kinect v2 real, nos quais foi identificado e corrigido um defeito de calibração que restringia a área projetada a uma pequena fração da janela, correção formalizada pela introdução de uma matriz de translação T_{shift} ao centro lógico da mesa.

Palavras-chave: realidade aumentada; modelo digital de elevação; kinect; calibração geométrica; RANSAC; treinamento militar.

ABSTRACT

This work presents the design and implementation of an Augmented Reality Sandbox system for military terrain-operations training, developed to meet an actual demand from the Simulation Section of the Agulhas Negras Military Academy (AMAN). The system integrates a Microsoft Kinect v2 depth sensor, a conventional projector, and a reference Digital Elevation Model (DEM), providing real-time visual feedback — **red** for excess sand, **blue** for shortage, and **green** for conformity — projected directly onto the sand surface. The Kinect-to-table geometric calibration is obtained empirically and automatically from a flat reference surface (a lid placed over the box or the empty base itself): since the sensor’s field of view is wider than the box’s usable area, a RANSAC algorithm first isolates the largest set of coplanar points, discarding the frame, floor, and noise, after which the plane normal is refined by singular value decomposition (SVD) over those inliers alone; orthonormal bases are then built via Gram-Schmidt and combined into homogeneous affine transformations that place the zero elevation at the box’s wooden base. The table-to-projector mapping employs Tsai’s pinhole camera model. Rendering employs a 30×30 discretized grid with spatial aggregation for statistical noise filtering. The monolithic modular architecture features a four-tier fallback chain (PyKinect2, Open3D, libfreenect2, and interactive mouse-based simulation), ensuring operation even without hardware. The mathematical engine’s correctness is backed by an automated suite of 83 unit tests, all passing, covering every algebraic contract of the system. The system was validated both in simulation mode and in field trials with the real Kinect v2 sensor, during which a calibration defect that restricted the projected area to a small fraction of the window was identified and corrected, formalized by the introduction of a translation matrix T_{shift} to the logical table center.

Keywords: augmented reality; digital elevation model; kinect; geometric calibration; RANSAC; military training.

LISTA DE ILUSTRAÇÕES

Figura 1	– Máquina de estados do orquestrador.	29
Figura 2	– Interface gráfica de configuração inicial (CustomTkinter), aba “Calibração”: modo de calibração (tampa/base), distância do Kinect à superfície de referência, tolerância de aceitação e o fator inicial de zoom do projetor.	31
Figura 3	– Cadeia de <i>fallback</i> do <code>KinectSensor</code>	32
Figura 4	– Crescimento do número de testes unitários ao longo do histórico de <i>commits</i> do repositório que modificaram <code>test_motor_caixao.py</code> (dados extraídos de <code>git log</code>).	47
Figura 5	– Distribuição dos 83 testes unitários pelas cinco frentes funcionais do <i>pipeline</i> (mesmos dados da Tabela 5).	48
Figura 6	– Cobertura de linhas por módulo, medida com <code>coverage.py</code> sobre a execução completa da suíte de testes.	49
Figura 7	– Janela <code>Gabarito_MDE</code> : heatmap do MDE sintético Cubo Central, com o HUD de estado no canto superior/inferior esquerdo.	55
Figura 8	– Janela <code>Projecao_Areia</code> : malha discretizada renderizada pelo <i>pipeline</i> completo (<code>gerar_imagem_grade_cores</code>), em um estágio intermediário de modelagem — verde (correto), azul (falta areia) e vermelho (excesso) simultaneamente visíveis.	55
Figura 9	– Reconstituição do defeito de calibração (versão 3.0), obtida executando o próprio <i>pipeline</i> de renderização de produção (<code>gerar_imagem_grade_cores</code>) com os parâmetros <i>mock</i> de projetor então usados em modo real ($f_x = f_y = 500$, $c_x = 320$, $c_y = 240$, $\mathbf{t}_{\text{ext}} = (0, 0, 1)^T$): apenas uma fração da janela é preenchida pela malha, o restante permanece preto. Não é a captura de tela original do incidente de campo, que não foi preservada — é uma reprodução fiel do mecanismo do defeito a partir dos parâmetros documentados na versão 3.0 do código.	57

LISTA DE TABELAS

Tabela 1	–	Parâmetros intrínsecos do sensor de profundidade do Kinect v2.	20
Tabela 2	–	Posicionamento do presente trabalho frente a implementações revisadas.	26
Tabela 3	–	Responsabilidades dos módulos.	29
Tabela 4	–	Padrões de projeto aplicados.	31
Tabela 5	–	Estrutura da suíte de testes unitários, por frente funcional (estado atual).	45
Tabela 6	–	Testes unitários <i>versus</i> teste de ponta a ponta, na suíte atual.	50
Tabela 7	–	Roteiro de demonstração passo a passo.	64

LISTA DE ABREVIATURAS E SIGLAS

AR	<i>Augmented Reality</i> — Realidade Aumentada
BGR	<i>Blue, Green, Red</i> (ordem de canais do OpenCV)
CV	<i>Computer Vision</i> — Visão Computacional
FOV	<i>Field of View</i> — Campo de Visão
FPS	<i>Frames Per Second</i> — Quadros por Segundo
GIL	<i>Global Interpreter Lock</i> (Python)
GUI	<i>Graphical User Interface</i>
IME	Instituto Militar de Engenharia
IR	<i>Infrared</i> — Infravermelho
MDE	Modelo Digital de Elevação
OOP	<i>Object-Oriented Programming</i>
PFC	Projeto Final de Curso
RANSAC	<i>Random Sample Consensus</i>
RGB-D	<i>Red–Green–Blue + Depth</i>
SDK	<i>Software Development Kit</i>
SVD	<i>Singular Value Decomposition</i>
TDD	<i>Test-Driven Development</i>
USB	<i>Universal Serial Bus</i>

LISTA DE SÍMBOLOS

$\mathbf{p}_i$	i -ésimo ponto da nuvem capturada pelo sensor
$\bar{\mathbf{p}}$	centroide da nuvem de pontos
$\mathbf{n}$	vetor normal unitário do plano da mesa
T	matriz afim 4×4 de transformação Kinect $\rightarrow$ Mesa
T_{shift}	translação que desloca o centroide para o centro lógico da mesa
L_x, L_y	dimensões físicas da mesa nos eixos X e Y (m)
h_{max}	espessura máxima de areia mapeável (m)
N_x, N_y	número de células da malha nos eixos X e Y
$\Delta x, \Delta y$	tamanho linear de cada célula (m)
$\bar{Z}_{i,j}^{\text{real}}$	altura média da célula (i, j) medida pelo Kinect
$Z_{i,j}^{\text{MDE}}$	altura alvo da célula (i, j) extraída do MDE
τ	tolerância vertical para classificação de cor (m)
d_{in}	limiar de distância ao plano candidato para um ponto ser inlier no RANSAC (m)
K	matriz intrínseca do projetor
f_x, f_y	distâncias focais em pixels do projetor
α	intensidade da pá virtual (m por evento)
σ	desvio padrão do perfil gaussiano da pá virtual

SUMÁRIO

1	INTRODUÇÃO	16
1.1	CONTEXTO E MOTIVAÇÃO	16
1.2	DEFINIÇÃO DO PROBLEMA	17
1.3	OBJETIVOS	18
1.3.1	OBJETIVO GERAL	18
1.3.2	OBJETIVOS ESPECÍFICOS	18
1.4	CONTRIBUIÇÃO E JUSTIFICATIVA	18
1.5	ORGANIZAÇÃO DO DOCUMENTO	19
2	FUNDAMENTAÇÃO TEÓRICA	20
2.1	O SENSOR MICROSOFT KINECT	20
2.1.1	PRINCÍPIO DE MEDIÇÃO	20
2.1.2	PARÂMETROS INTRÍNSECOS	20
2.1.3	SENSORES ALTERNATIVOS: AZURE KINECT E INTEL REALSENSE	21
2.1.4	RUÍDO E TRATAMENTO	21
2.2	MODELO DE CÂMERA PINHOLE E CALIBRAÇÃO DE TSAI	22
2.3	ÁLGEBRA LINEAR COMPUTACIONAL	22
2.3.1	DECOMPOSIÇÃO EM VALORES SINGULARES	22
2.3.2	AMOSTRAGEM ROBUSTA: RANSAC	22
2.3.3	PROCESSO DE GRAM-SCHMIDT	23
2.3.4	TRANSFORMAÇÕES AFINS HOMOGÊNEAS	23
2.4	MODELOS DIGITAIS DE ELEVAÇÃO E O FORMATO GEOTIFF	23
3	TRABALHOS RELACIONADOS	25
3.1	AR SANDBOXES: ESTADO DA ARTE E ADOÇÃO ACADÊMICA	25
3.2	POSICIONAMENTO DO PRESENTE TRABALHO	26
4	ARQUITETURA DE SOFTWARE	27
4.1	ESTILO ARQUITETURAL: MONOLÍTICO MODULARIZADO	27
4.1.1	INSTALADOR <i>OFFLINE</i> PARA WINDOWS	27
4.2	MÓDULOS DO SISTEMA	28
4.3	MÁQUINA DE ESTADOS FINITA	29
4.4	SAÍDAS VISUAIS	30
4.4.1	SAÍDA TEMPORÁRIA DE CALIBRAÇÃO: O TABULEIRO QUADRICULADO	30
4.5	PADRÕES DE PROJETO APLICADOS	30
4.5.1	CADEIA DE <i>FALLBACK</i> DO SENSOR	32

5	MOTOR MATEMÁTICO	34
5.1	PASSO 1 — AJUSTE DE PLANO: RANSAC SEGUIDO DE SVD	34
5.1.1	ISOLAMENTO DE INLIERS POR RANSAC	34
5.1.2	REFINAMENTO POR SVD	34
5.2	PASSO 2 — CONSTRUÇÃO DA BASE DA MESA	35
5.3	PASSO 3 — TRANSFORMAÇÃO AFIM 4×4	36
5.3.1	COMPOSIÇÃO COM T_{shift} — CENTRO LÓGICO E COTA ZERO NA BASE . .	36
5.4	PASSO 4 — PROJEÇÃO 3D→2D PELO MODELO DE TSAI	37
5.4.1	CALIBRAÇÃO EMPÍRICA ALTERNATIVA DO PROJETOR (TABULEIRO DE XADREZ)	37
5.5	DISCUSSÃO — RANSAC E SVD: UMA COMPOSIÇÃO DELIBERADA . .	38
6	RENDERIZAÇÃO POR MALHA DISCRETIZADA E EMULADOR INTERATIVO	40
6.1	MOTIVAÇÃO PARA A DISCRETIZAÇÃO EM MALHA	40
6.2	DEFINIÇÃO FORMAL DA MALHA	40
6.3	AGREGAÇÃO ESPACIAL E FILTRAGEM DE RUÍDO	41
6.4	REGRAS DE COLORAÇÃO	41
6.5	PROJEÇÃO EM LOTE E RASTERIZAÇÃO	41
6.6	EMULADOR INTERATIVO DE AREIA	42
6.6.1	ESTADO PERSISTENTE DA AREIA VIRTUAL	42
6.6.2	MODELO GAUSSIANO DA PÁ VIRTUAL	43
6.6.3	MAPEAMENTO <i>PIXEL</i> →COORDENADA FÍSICA	43
7	RESULTADOS E VALIDAÇÃO	44
7.1	ESTRATÉGIA DE TESTES AUTOMATIZADOS	44
7.1.1	FILOSOFIA: TRANSPARÊNCIA MATEMÁTICA	45
7.1.2	TESTE DE ACEITAÇÃO DE PONTA A PONTA	46
7.2	EVIDÊNCIAS QUANTITATIVAS DE CORRETEDE E REPRODUTIBILIDADE	46
7.2.1	FUNCIONAMENTO DOS TESTES UNITÁRIOS	47
7.2.2	COBERTURA DE CÓDIGO: QUANTO DO SISTEMA É REALMENTE EXERCI- TADO	48
7.2.3	TESTES UNITÁRIOS <i>VERSUS</i> TESTES DE PONTA A PONTA	49
7.2.4	REPRODUTIBILIDADE INDEPENDENTE	52
7.2.5	LIMITAÇÃO DA VALIDAÇÃO FÍSICA: CONEXÃO DO KINECT E CONSTRU- ÇÃO DO CAIXÃO DE TESTE	52
7.3	VALIDAÇÃO FUNCIONAL DO <i>PIPELINE</i>	53
7.3.1	MODOS DE SIMULAÇÃO	54
7.3.2	MODOS DE REAL	54
7.4	DEFEITO DETECTADO E CORREÇÃO IMPLEMENTADA	56

7.4.1	DIAGNÓSTICO	56
7.4.2	CORREÇÃO IMPLEMENTADA	57
7.5	DISCUSSÃO DE <i>TRADE-OFFS</i>	58
7.5.1	RESOLUÇÃO DA MALHA	58
7.5.2	TOLERÂNCIA DE COR	58
7.5.3	LINGUAGEM E <i>STACK</i>	58
8	CONCLUSÃO	59
8.1	SÍNTESE DAS CONTRIBUIÇÕES	59
8.2	ATENDIMENTO AOS OBJETIVOS	60
8.3	TRABALHOS FUTUROS	60
8.4	CONSIDERAÇÕES FINAIS	61
	REFERÊNCIAS	62
	APÊNDICE A – ROTEIRO DE DEMONSTRAÇÃO PARA A BANCA	64
	ANEXO A – REPOSITÓRIO DE CÓDIGO-FONTE	66

1 INTRODUÇÃO

1.1 Contexto e Motivação

A leitura e a interpretação de cartas topográficas constituem competências fundamentais para o oficial combatente. A capacidade de visualizar, em três dimensões, o relevo de uma região de operações é determinante para o planejamento de manobras, o posicionamento de armas de apoio, a escolha de itinerários de ataque e a definição de bases de fogo. Tradicionalmente, esse treinamento é apoiado por dois recursos complementares: a **carta topográfica em papel**, com curvas de nível, e o **caixão de areia físico**, no qual a turma de instrução modela manualmente o terreno a partir da carta para obter uma representação tridimensional tangível.

Embora o caixão de areia seja um instrumento didático consagrado, a transição cognitiva entre a representação bidimensional da carta (curvas de nível) e o relevo tridimensional construído na areia exige considerável esforço mental e tempo de instrução. Erros de interpretação das curvas de nível resultam em terrenos modelados que divergem da realidade cartográfica, comprometendo a fidelidade do exercício.

Os **caixões de areia com realidade aumentada** — na literatura internacional, *Augmented Reality Sandboxes* — emergiram, na última década, como uma resposta tecnológica a esse problema. Por meio da combinação de um **sensor de profundidade** (tipicamente o Microsoft Kinect) com um **projektor multimídia** suspenso sobre a caixa, esses sistemas projetam, em tempo real, curvas de nível coloridas, escalas batimétricas ou texturas de relevo diretamente sobre a superfície da areia. À medida que o operador modifica o relevo — cavando ou acumulando areia com as mãos — a projeção se ajusta dinamicamente, fornecendo *feedback* visual imediato e dispensando a interpretação mental das curvas de nível.

A iniciativa pioneira do projeto SARndbox, da Universidade da Califórnia em Davis (KREYLOS, 2016), demonstrou a viabilidade técnica da abordagem. Adaptações posteriores expandiram o conceito para a educação em geociências, hidrologia e cartografia. Neste trabalho, por sugestão da orientação, evita-se o uso do anglicismo como apelido do sistema desenvolvido, preferindo-se, ao longo de todo o texto, a denominação em língua portuguesa “caixão de areia com realidade aumentada”.

No contexto militar, entretanto, três limitações dos sistemas existentes restringem sua adoção:

1. A maioria dos sistemas projeta apenas curvas de nível decorativas ou simulações

- hidrológicas qualitativas, sem comparação com um relevo de referência específico;
2. Não há guia visual explícito indicando onde cavar ou onde acumular areia para reproduzir um terreno alvo determinado pela cartografia oficial;
 3. As implementações de referência são frágeis quanto a falhas de hardware: a ausência do Kinect ou do arquivo de terreno provoca o encerramento abrupto do programa, comprometendo apresentações didáticas e demonstrações.

Este trabalho tem origem em uma demanda operacional concreta: a **Seção de Simulação da Academia Militar das Agulhas Negras (AMAN)** mantém, em sua **Área de Emissão de Ordens Tecnológicas**, um caixão de areia físico empregado rotineiramente na instrução de leitura de carta topográfica, no planejamento de operações e em exercícios de Comando e Estado-Maior. Nesse ambiente, o relevo é modelado manualmente pelos instruídos a partir de cartas em papel, e a aderência do terreno resultante à cartografia oficial é verificada apenas visualmente pelo instrutor, sem qualquer retorno quantitativo durante a modelagem. A Seção de Simulação identificou nesse processo uma oportunidade de modernização: dotar o caixão de areia já existente de um mecanismo de *feedback* automático, capaz de comparar o relevo modelado com um Modelo Digital de Elevação de referência sem substituir a mesa física nem alterar a rotina de instrução já consolidada. Esse é o contexto institucional que orienta os requisitos e as decisões de projeto apresentadas neste documento.

1.2 Definição do Problema

Diante das limitações expostas, este Projeto Final de Curso propõe a seguinte questão de pesquisa:

“Como projetar e implementar um sistema de Realidade Aumentada para caixão de areia que (a) compare quantitativamente, em tempo real, o relevo modelado com um Modelo Digital de Elevação (MDE) cartográfico de referência, (b) forneça feedback visual inequívoco indicando onde adicionar ou remover areia para minimizar essa diferença, e (c) opere de forma resiliente, sem qualquer falha catastrófica, mesmo na ausência do sensor de profundidade ou do arquivo de terreno?”

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver um sistema funcional de Caixão de Areia com Realidade Aumentada que projete sobre uma caixa física de areia uma **malha discretizada de quadrados coloridos** comparando, célula a célula, o relevo medido por um sensor de profundidade com o relevo de referência extraído de um arquivo GeoTIFF, e que opere em modalidade *plug & play* com fallback automático para um emulador interativo.

1.3.2 Objetivos específicos

1. Projetar uma arquitetura de software **monolítica modularizada** de três camadas (Hardware, Lógica, Dados), orquestrada por uma máquina de estados explícita;
2. Implementar o módulo `KinectSensor` com mecanismo de *cascading fallback* (`PyKinect2` $\rightarrow$ `Open3D` $\rightarrow$ `freenect` $\rightarrow$ Simulação);
3. Implementar o motor matemático `motor_caixao_areia.py` contemplando ajuste de plano por SVD, base ortonormal por Gram-Schmidt, transformação afim 4×4 e projeção *pinhole* de Tsai;
4. Implementar o adaptador `AdaptadorMDE` para leitura de GeoTIFF via `rasterio` com interpolação bilinear, e fallback para superfície gaussiana sintética;
5. Implementar discretização espacial em **malha regular**, com agregação de pontos por célula (filtragem estatística do ruído do sensor) e renderização por `cv2.fillPoly`;
6. Implementar um **emulador interativo** controlado por *mouse* que permita cavar e preencher a areia virtual quando o Kinect não estiver disponível;
7. Validar todo o motor matemático através de uma suíte de **testes unitários automatizados**, seguindo os princípios do *Test-Driven Development* (TDD);
8. Documentar o projeto em três níveis: documentação oficial acadêmica, README de uso prático e relatório de PFC em conformidade com as normas da ABNT.

1.4 Contribuição e Justificativa

A contribuição central deste trabalho reside na integração inédita de três elementos: (i) o **rigor matemático** de uma calibração geométrica completa (SVD + Gram-Schmidt + Tsai), (ii) a **regra de negócio quantitativa** de comparação com tolerância numérica explícita contra um MDE oficial, e (iii) uma **arquitetura resiliente** que prioriza a confiabilidade operacional na presença de falhas de hardware ou de dados. Diferentemente

de implementações que utilizam apenas mapas de cores qualitativos, o sistema proposto produz uma decisão operacional para cada célula da mesa: cavar, preencher ou manter.

A justificativa militar é direta: a doutrina nacional prescreve o uso do caixão de areia para o ensino e o planejamento tático em todas as escolas de formação. Reduzir o tempo necessário à modelagem do relevo e aumentar a fidelidade cartográfica do exercício produz ganho direto em horas de instrução e em qualidade do treinamento.

1.5 Organização do Documento

Os capítulos seguintes acompanham o próprio *pipeline* do sistema: da teoria (Capítulo 2), passando pelo posicionamento deste trabalho frente a implementações correlatas (Capítulo 3), e da arquitetura de *software* (Capítulo 4) à formalização matemática da calibração e da projeção (Capítulo 5), à renderização e ao emulador interativo (Capítulo 6) e, por fim, aos resultados experimentais e à conclusão. O sumário e a lista de figuras, nas páginas pré-textuais, detalham a numeração completa de seções; o Apêndice A reúne o roteiro de demonstração utilizado na defesa do PFC.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos teóricos fundamentais que sustentam o sistema desenvolvido, organizados em quatro blocos: o sensor de profundidade Kinect (Seção 2.1), o modelo de câmera *pinhole* estendido por Tsai (Seção 2.2), as ferramentas de álgebra linear computacional empregadas (Seção 2.3) e a representação cartográfica por Modelos Digitais de Elevação (Seção 2.4).

2.1 O Sensor Microsoft Kinect

O Microsoft Kinect é um dispositivo de captura tridimensional originalmente concebido para o controle gestual do videogame Xbox 360 (versão v1, 2010) e posteriormente sucedido pela versão para Xbox One (versão v2, 2014). Para fins acadêmicos e industriais, o sensor revelou-se notável pela combinação de baixo custo e qualidade aceitável de profundidade, tendo se consolidado como plataforma de pesquisa em visão computacional, robótica e interação humano-computador (ZHANG, 2012).

2.1.1 Princípio de medição

A versão v1 do Kinect mede a profundidade pelo princípio de luz estruturada: um projetor infravermelho emite um padrão pseudo-aleatório de pontos sobre a cena, e uma câmera infravermelha calibrada captura a deformação desse padrão. A profundidade de cada *pixel* é estimada por triangulação. A versão v2 substitui esse princípio pelo *Time-of-Flight* (ToF), no qual a profundidade é calculada a partir do deslocamento de fase de pulsos infravermelhos modulados (SELL; O'CONNOR, 2014).

2.1.2 Parâmetros intrínsecos

Para a versão v2, empregada como referência neste trabalho, os parâmetros intrínsecos do sensor de profundidade são:

Tabela 1 – Parâmetros intrínsecos do sensor de profundidade do Kinect v2.

Parâmetro	Valor
Resolução	512×424 pixels
Distância focal f_x, f_y	367,35 pixels
Centro óptico (c_x, c_y)	(260,00, 205,00) pixels
Faixa útil de operação	0,5 a 4,5 metros
Saída do SDK	milímetros (<code>uint16</code>)

A conversão entre pixel (u, v) e ponto 3D (X, Y, Z) no referencial do sensor usa a relação canônica de *back-projection* pinhole:

$$X = (u - c_x) \cdot \frac{Z}{f_x}, \quad Y = (v - c_y) \cdot \frac{Z}{f_y}, \quad Z = \frac{d_{\text{mm}}}{1000}, \quad (2.1)$$

onde d_{mm} é a profundidade lida pelo sensor.

2.1.3 Sensores alternativos: Azure Kinect e Intel RealSense

O Microsoft Kinect v2 foi descontinuado pela fabricante em 2015, o que motivou a avaliação de sensores substitutos para a continuidade da linha de pesquisa. O **Azure Kinect DK**, sucessor direto lançado em 2019, mantém o princípio *Time-of-Flight* da versão v2, porém com maior resolução de profundidade (1024×1024 no modo *narrow field of view*) e uma unidade de medição inercial (IMU) integrada, útil para compensar pequenas oscilações do suporte do sensor. A família **Intel RealSense** (modelos D400), por sua vez, emprega visão estéreo ativa: um par de câmeras infravermelhas associado a um projetor de padrão estrutural estima a profundidade por triangulação, dispensando o cálculo de fase do *Time-of-Flight* e reduzindo o consumo energético, ao custo de maior ruído em superfícies pouco texturizadas — não é o caso da areia, cuja textura granular favorece a correspondência estéreo.

Ambos os sensores expõem SDKs distintos do PyKinect2 utilizado para o Kinect v2, mas convergem para o formato de nuvem de pontos RGB-D consumido pela biblioteca **Open3D** (ZHOU; PARK; KOLTUN, 2018), já integrada à cadeia de *fallback* do sistema (Seção 4.5.1) como segundo nível, imediatamente abaixo do PyKinect2. Na prática, isso significa que a migração do Kinect v2 para um desses sensores mais recentes exige apenas a captura da nuvem no formato esperado por **Open3D** — todo o restante do *pipeline* matemático (Capítulo 5), agnóstico ao fabricante do sensor, permanece inalterado. Essa possibilidade é retomada como trabalho futuro na Conclusão.

2.1.4 Ruído e tratamento

O sensor apresenta ruído gaussiano cujo desvio padrão cresce quadraticamente com a profundidade. Na faixa de operação adotada (2–3 m, com o sensor a 2,5 m da mesa), valores típicos situam-se entre $\sigma \approx 2$ e $\sigma \approx 4$ mm por *pixel*. Esse ruído justifica a estratégia de **agregação espacial** (Capítulo 6): ao se calcular a média de n pontos por célula da malha, o desvio padrão do estimador da altura é reduzido para $\sigma_{\bar{Z}} = \sigma/\sqrt{n}$, conforme o *Central Limit Theorem*.

2.2 Modelo de Câmera Pinhole e Calibração de Tsai

A projeção de um ponto 3D no plano de imagem 2D segue, em primeira aproximação, o modelo *pinhole*:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot \begin{bmatrix} R_{\text{ext}} & \mathbf{t}_{\text{ext}} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad (2.2)$$

onde K é a **matriz intrínseca** do dispositivo (câmera ou projetor), R_{ext} e $\mathbf{t}_{\text{ext}}$ formam a **matriz extrínseca** (rotação e translação do referencial mundo para o referencial do dispositivo) e s é um fator de escala não nulo.

O modelo de Tsai Tsai (1987) estende o *pinhole* canônico com a inclusão de **distorções óticas** radiais (coeficientes k_1, k_2, k_3) e tangenciais (p_1, p_2), fornecendo precisão sub-pixel para câmeras e projetores reais. A biblioteca OpenCV (BRADSKI; KAEHLER, 2008) implementa esse modelo através das funções `cv2.calibrateCamera`, `cv2.solvePnP` e `cv2.projectPoints`, todas utilizadas no presente trabalho.

2.3 Álgebra Linear Computacional

2.3.1 Decomposição em Valores Singulares

Dada uma matriz $M \in \mathbb{R}^{N \times 3}$, sua decomposição em valores singulares é $M = U\Sigma V^T$, com Σ diagonal contendo $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. O vetor singular direito associado ao menor valor σ_3 , isto é, a última linha de V^T , constitui a solução do problema de mínimos quadrados restrito

$$\mathbf{n}^* = \arg \min_{\|\mathbf{v}\|=1} \|M\mathbf{v}\|_2^2, \quad (2.3)$$

o que, no contexto deste trabalho, corresponde ao vetor normal do plano que melhor se ajusta à nuvem de pontos centralizada (Seção 5.1).

2.3.2 Amostragem Robusta: RANSAC

O ajuste de plano por SVD (Seção 2.3.1) é um estimador de **mínimos quadrados**: minimiza a soma dos quadrados das distâncias de *todos* os pontos da nuvem ao plano, e por isso é sensível a *outliers* — pontos que não pertencem à superfície de interesse. No caso de campo deste trabalho, o campo de visão do Kinect é mais largo que a área útil do caixão de areia, de modo que a nuvem capturada durante a calibração inclui, além da superfície de referência, a moldura de madeira do caixão, o piso ao redor e ruído da sala — outliers que, se incluídos ingenuamente no SVD, distorcem a normal calculada.

O algoritmo **RANSAC** (*Random Sample Consensus*), proposto por Fischler e Bolles (1981), resolve esse problema por amostragem aleatória repetida. Dada uma nuvem de N pontos, cada iteração executa três passos: (i) sorteiam-se k pontos mínimos necessários para definir o modelo — no caso de um plano, $k = 3$; (ii) constrói-se o modelo candidato a partir desses pontos; (iii) conta-se o número de **inliers** — pontos da nuvem inteira cuja distância ao modelo candidato é inferior a um limiar d_{in} . Após um número fixo de iterações, o modelo com maior número de inliers é escolhido como vencedor. Diferentemente do SVD, o RANSAC não assume que a maioria dos pontos seja bem descrita por um único modelo suave; ele explicitamente *separa* a nuvem em inliers e outliers, sendo tolerante a frações significativas de dados espúrios, desde que o conjunto de inliers permaneça majoritário dentro da amostra observada.

Neste trabalho, o RANSAC não substitui o SVD, mas o **precede**: o plano vencedor do RANSAC delimita o conjunto de inliers (presumivelmente a superfície de referência plana), e apenas esses pontos alimentam o refinamento por mínimos quadrados via SVD, que produz a normal final com precisão sub-milimétrica. Essa combinação — robustez do RANSAC seguida da precisão do SVD sobre os inliers selecionados — é detalhada, com a formalização algébrica completa e sua aplicação à calibração Kinect → Mesa, na Seção 5.1.

2.3.3 Processo de Gram-Schmidt

O processo de ortogonalização de Gram-Schmidt converte um conjunto linearmente independente $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ em um conjunto ortonormal preservando os subespaços gerados. Neste trabalho, o procedimento é aplicado para construir uma base ortonormal $\{X_{\text{mesa}}, Y_{\text{mesa}}, Z_{\text{mesa}}\}$ a partir do vetor normal obtido por SVD.

2.3.4 Transformações afins homogêneas

Uma transformação afim 3D é representada de forma compacta por uma matriz 4×4 que combina rotação $R \in \mathbb{R}^{3 \times 3}$ e translação $\mathbf{t} \in \mathbb{R}^3$:

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad \begin{bmatrix} \mathbf{p}' \\ 1 \end{bmatrix} = T \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}. \quad (2.4)$$

A composição de transformações é dada pelo produto matricial. Em particular, a aplicação sucessiva de T (alinhamento) e T_{shift} (deslocamento ao centro lógico da mesa) é realizada por $T_{\text{final}} = T_{\text{shift}} \cdot T$, conforme discutido na Seção 5.3.1.

2.4 Modelos Digitais de Elevação e o Formato GeoTIFF

Um **Modelo Digital de Elevação** (MDE) é uma representação matricial discreta da altimetria de uma região, na qual cada *pixel* codifica a elevação Z correspondente a

uma coordenada geográfica (X, Y) (LI; ZHU; GOLD, 2005). MDEs constituem produto de saída fundamental da Geomática e são amplamente empregados em planejamento militar, hidrografia, agricultura de precisão e simulação ambiental.

O formato **GeoTIFF** (RITTER; RUTH et al., 2019) é um padrão aberto que estende o tradicional **TIFF** com metadados de georreferenciamento: sistema de coordenadas, *datum*, projeção cartográfica e transformação afin que mapeia índices de *pixel* para coordenadas do mundo. A leitura programática do formato em Python é realizada pela biblioteca **rasterio** (GILLIES et al., 2013–2024), que retorna um arranjo NumPy associado a metadados consultáveis. A consulta da elevação em coordenadas arbitrárias entre os pixels é tratada neste trabalho por **interpolação bilinear**, implementada na classe `scipy.interpolate.RegularGridInterpolator` (VIRTANEN et al., 2020).

3 TRABALHOS RELACIONADOS

Este capítulo situa o presente trabalho no panorama de sistemas de caixão de areia com realidade aumentada, revisando a origem do conceito e sua adoção acadêmica (Seção 3.1) e posicionando explicitamente as contribuições deste PFC frente a esse estado da arte (Seção 3.2). A diversificação de sensores de profundidade empregados por essas implementações já foi discutida na Seção 2.1.3.

3.1 AR Sandboxes: Estado da Arte e Adoção Acadêmica

O conceito de caixão de areia com realidade aumentada foi consolidado pelo projeto **SARndbox**, desenvolvido por Kreylos Kreylos (2016) no KeckCAVES da Universidade da Califórnia em Davis. Escrito em C++/OpenGL para aproveitar processamento gráfico dedicado, o SARndbox captura a superfície da areia com um Kinect, projeta escalas hipsométricas e curvas de nível coloridas em tempo real e simula, sobre o relevo físico, o escoamento de água por um solver de dinâmica de fluidos. Mais de 150 instalações seguindo esse modelo foram documentadas em escolas, universidades, museus e centros de pesquisa, e a própria página do projeto lista, entre os domínios de aplicação já explorados pela comunidade, hidrologia, geologia, mineração, planejamento regional e **operações militares** (KREYLOS, 2016) — ainda que, na revisão realizada para este trabalho, não tenham sido encontradas implementações publicamente documentadas que combinem esse último domínio com comparação quantitativa a um Modelo Digital de Elevação cartográfico oficial, lacuna que motiva a Seção 3.2.

A validade pedagógica da abordagem foi avaliada empiricamente por Woods et al. Woods et al. (2016), em um estudo piloto que empregou o SARndbox em laboratórios introdutórios de Geologia para ensinar leitura de mapas topográficos e processos superficiais, relatando ganhos observados na capacidade de visualização espacial dos estudantes. Esse resultado é relevante para o presente trabalho por oferecer evidência prévia, independente e publicada, de que a transição cognitiva entre carta topográfica e relevo tridimensional tangível — o problema central motivador deste PFC (Seção 1.1) — é de fato facilitada pela metáfora do caixão de areia aumentado, dispensando a necessidade de este trabalho revalidar esse pressuposto pedagógico isoladamente.

Mais recentemente, Wellmann et al. Wellmann et al. (2022) propuseram o **Open AR-Sandbox**, uma reimplementação de código aberto em Python voltada à visualização de informação geológica de subsuperfície (camadas de litologia, não apenas topografia), com interface modularizável para uso em pesquisa e divulgação científica. A escolha por Python, em vez do C++ nativo do SARndbox, aproxima essa implementação do ecossistema cientí-

fico também adotado neste trabalho (NumPy, OpenCV — Seção 7.5), reforçando que a linguagem não é mais um fator limitante de desempenho para esse tipo de aplicação em hardware convencional.

3.2 Posicionamento do Presente Trabalho

A Tabela 2 resume as diferenças entre as implementações revisadas e o sistema desenvolvido neste PFC, ao longo de quatro eixos de comparação.

Tabela 2 – Posicionamento do presente trabalho frente a implementações revisadas.

Eixo	SARndbox e variantes revisadas	Este trabalho
<i>Feedback</i> ao operador	Curvas de nível, escalas hipsométricas ou camadas geológicas — qualitativo, sem alvo externo definido	Decisão célula a célula (cavar/preencher/manter) por comparação quantitativa com tolerância explícita contra um MDE de referência
Resiliência a falta de hardware	Não documentada nas publicações revisadas; presume-se sensor e GPU dedicada disponíveis	Cadeia de <i>fallback</i> em quatro níveis (Seção 4.5.1), com simulação interativa completa na ausência de qualquer sensor
Origem do MDE de referência	Nenhuma (SARndbox, Woods et al.) ou dado geológico de subsuperfície sem vínculo cartográfico oficial (Open AR-Sandbox)	Arquivo GeoTIFF georreferenciado fornecido pelo operador — compatível com produtos oficiais como os do BDGEx (Seção 2.4) —, com <i>fallback</i> sintético para demonstração sem conectividade
Domínio de aplicação	Educação em geociências, hidrologia e divulgação científica	Treinamento militar em leitura de carta e planejamento tático, em atendimento a uma demanda real da Seção de Simulação da AMAN

A contribuição deste trabalho não está, portanto, em propor a técnica de caixão de areia com realidade aumentada — consolidada desde o SARndbox e validada pedagogicamente por Woods et al. Woods et al. (2016) —, mas em preencher as três lacunas já enunciadas na Seção 1.1: substituir o retorno visual qualitativo por uma regra de negócio quantitativa contra um MDE cartográfico oficial, tornar explícita a decisão operacional célula a célula, e assegurar resiliência operacional total diante da ausência de hardware — nenhuma das quais documentada em conjunto nas implementações revisadas neste capítulo.

4 ARQUITETURA DE SOFTWARE

Este capítulo descreve a arquitetura do sistema, organizada em cinco seções: a justificativa do estilo arquitetural adotado (Seção 4.1), a estrutura de módulos (Seção 4.2), a máquina de estados que orquestra a execução (Seção 4.3), as saídas visuais apresentadas ao operador (Seção 4.4) e os padrões de projeto aplicados (Seção 4.5).

4.1 Estilo Arquitetural: Monolítico Modularizado

A arquitetura adotada é **monolítica modularizada**: todos os módulos executam dentro de um único processo Python, comunicando-se por chamada direta de função. A escolha foi norteadada por três requisitos do domínio:

Latência mínima. O *loop* de Realidade Aumentada opera em mais de 30 quadros por segundo, e qualquer serialização ou marshalling intermediário (como em uma arquitetura de microsserviços) introduziria atraso intolerável entre a captura e a projeção.

Simplicidade de *deploy*. Um único ponto de entrada (`main.py`) e um conjunto fixo de dependências permitem operação *plug & play* em qualquer máquina Windows ou Linux com Python e OpenCV instalados — condição explorada ao máximo pelo instalador offline descrito na Seção 4.1.1.

Testabilidade. Cada módulo encapsula uma responsabilidade bem definida, permitindo testes unitários isolados sem necessidade de *mock* de rede ou *containers*.

4.1.1 Instalador *offline* para Windows

A operação em unidades de instrução militar não pode presumir a presença de pessoal com formação em desenvolvimento de *software* capaz de configurar um ambiente Python do zero, tampouco conectividade constante à internet para `pip install`. Para eliminar essa barreira de adoção, o projeto inclui, na pasta `installer/` do repositório (Anexo A), um instalador único e **100% offline** para Windows 10/11 de 64 *bits*, gerado com o Inno Setup a partir do script `ARSandbox.iss`. Em uma máquina limpa, sem qualquer pré-requisito além do próprio Windows, o instalador executa automaticamente, em sequência:

1. Copia o código-fonte, o mapa GeoTIFF de exemplo e a documentação para `C:\ARSandbox`;

2. Instala o Python 3.12 (reaproveitando uma instalação existente, se houver) e cria um ambiente virtual dedicado;
3. Instala todas as dependências (NumPy, OpenCV, **rasterio**, SciPy, CustomTkinter, **comtypes**, **pykinect2**) a partir de *wheels* pré-baixadas, embutidas no próprio instalador — nenhum acesso à internet é necessário nesta etapa;
4. Aplica automaticamente, via **patch_pykinect2.py**, os cinco patches de compatibilidade que permitem ao **pykinect2** — escrito originalmente para Python 2/3.6 — funcionar em Python 3.12 (entre eles, a correção de **sizeof(tagSTATSTG)** e a substituição de **numpy.distutils**, descontinuado nas versões recentes do NumPy);
5. Instala o Kinect for Windows Runtime v2.0, necessário para o driver do sensor mesmo fora de um ambiente de desenvolvimento;
6. Executa um teste de fumaça dos módulos importados, registrando o resultado em um arquivo de *log*, e cria os atalhos **AR Sandbox** e **AR Sandbox — Diagnóstico do Kinect** no Menu Iniciar.

O resultado é que um instrutor da Seção de Simulação da AMAN, sem qualquer conhecimento prévio de Python, consegue colocar o sistema em operação executando um único instalador e clicando em “Avançar” — o mesmo princípio de *plug & play* que orienta a cadeia de *fallback* do sensor (Seção 4.5.1) aplicado, aqui, à própria instalação do *software*.

4.2 Módulos do Sistema

A árvore de arquivos do projeto é apresentada a seguir:

```

1 PFC-2026/
2 --- main.py                # Orquestrador (FSM + mouse callback)
3 --- kinect_sensor.py       # Camada de Hardware (KinectSensor)
4 --- motor_caixao_areia.py  # Motor matematico (algebra + grade)
5 --- mde_cartografia.py     # Adaptador de MDE (GeoTIFF + fallback)
6 --- test_motor_caixao.py   # 83 testes unitarios automatizados
7 --- DOCUMENTACAO_OFICIAL.md # Documentacao academica completa
8 --- README.md              # Guia de uso pratico
9 --- relatorio_tcc/         # Esta pasta (relatorio LaTeX do PFC)

```

Listing 4.1 – Estrutura de módulos do projeto.

A Tabela 3 resume a responsabilidade de cada módulo.

Tabela 3 – Responsabilidades dos módulos.

Camada	Módulo	Responsabilidade
Hardware	kinect_sensor.py	Captura da nuvem 3D, fallback, simulação
Lógica	motor_caixao_areia.py	Álgebra linear, discretização, projeção
Dados	mde_cartografia.py	Leitura de GeoTIFF, fallback gaussiano
Orquestração	main.py	FSM, janelas, callback de <i>mouse</i>

4.3 Máquina de Estados Finita

A execução do sistema é controlada por uma máquina de estados finita (*Finite State Machine* — FSM) com quatro estados principais e transições determinísticas controladas por teclado. A Figura 1 ilustra o diagrama de estados.

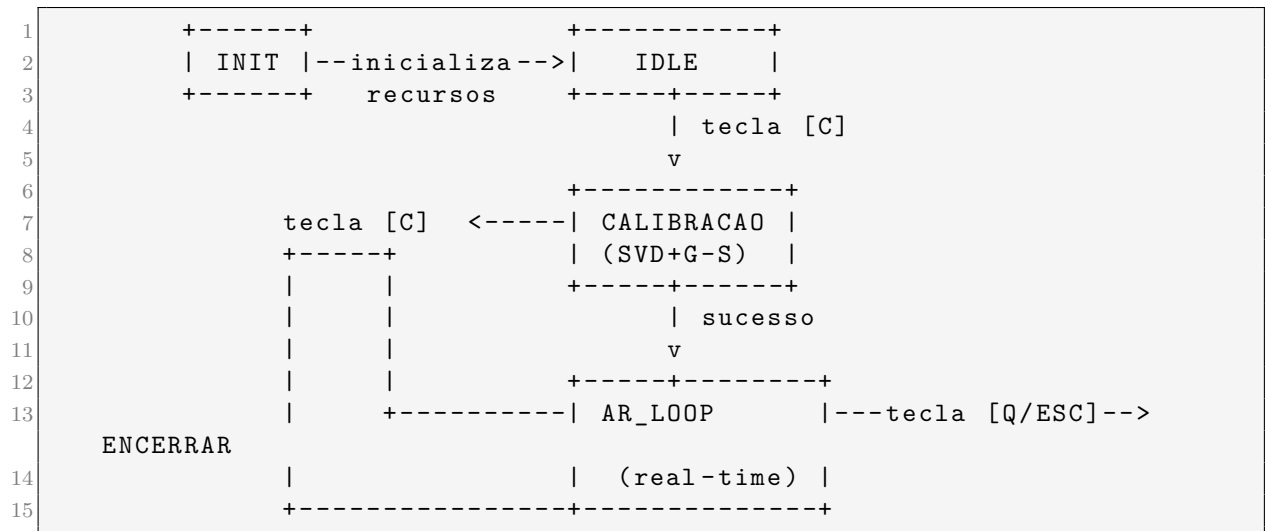


Figura 1 – Máquina de estados do orquestrador.

INIT. Inicializa o `KinectSensor` (com cadeia de *fallback*), carrega o `AdaptadorMDE` (com *fallback* gaussiano), cria as duas janelas OpenCV e registra o *callback* de *mouse*.

IDLE. Exibe o mapa de profundidade colorido enquanto aguarda o comando de calibração (tecla C). Permite encerramento imediato.

CALIBRACAO. Em modo real, captura uma nuvem de pontos, executa SVD + Gram-Schmidt + montagem de T + composição com T_{shift} (Seção 5.3.1). Em modo simulação, atribui $T = I_4$ e configura a projeção que mapeia $[0, L_x] \times [0, L_y]$ na janela do projetor.

AR_LOOP. Executa, a cada *frame*, o *pipeline* completo: captura $\rightarrow$ transformação $\rightarrow$ discretização $\rightarrow$ comparação MDE $\rightarrow$ projeção Tsai $\rightarrow$ rasterização. Eventos de *mouse* entre quadros modificam a grade de areia virtual (modo simulação).

4.4 Saídas Visuais

O sistema apresenta ao operador **duas janelas OpenCV** simultâneas, com propósitos distintos e complementares:

Projecao_Areia. Destinada ao **projektor** físico suspenso sobre a mesa: exhibe exclusivamente a malha contínua de quadrados coloridos (vermelho/azul/verde) que constitui o *feedback* operacional. Nenhum texto ou elemento de interface é desenhado sobre essa janela durante a operação normal, para não contaminar a projeção sobre a areia.

Gabarito_MDE. Janela de **controle do operador**, exibida no monitor do computador (não no projektor). Mostra o *heatmap* do MDE de referência que está sendo replicado, sobreposto a um HUD (*Heads-Up Display*) semitransparente com a legenda de cores, o estado corrente da máquina de estados, o indicador de calibração (pendente, em cache ou recém-calculada) e a taxa de quadros por segundo.

4.4.1 Saída temporária de calibração: o tabuleiro quadriculado

Durante a etapa opcional de calibração empírica do projektor (via `calibrar_projektor`, Seção 5.4.1), a janela **Projecao_Areia** deixa **temporariamente** de exibir a malha de cores e passa a projetar um **padrão de tabuleiro de xadrez** (`gerar_imagem_tabuleiro`) sobre a superfície de referência. Essa saída é usada apenas enquanto dura a rotina de calibração: uma câmera captura o tabuleiro projetado, a função `encontrar_cantos_tabuleiro` detecta os cantos internos na imagem capturada e `cv2.calibrateCamera` estima, por otimização não linear, os parâmetros intrínsecos do projektor a partir das correspondências entre cantos conhecidos e cantos detectados. Um *trackbar* de ajuste do fator de zoom/*throw ratio* (“Zoom proj x100”) é exibido na janela **Gabarito_MDE** durante essa etapa, permitindo ao operador corrigir o chute inicial da distância focal caso a otimização convirja para um mínimo local incorreto. Encerrada a calibração — com sucesso ou por cancelamento —, a janela **Projecao_Areia** volta a exibir exclusivamente a malha de cores. Este é, portanto, um mecanismo de **depuração e calibração assistida**, e não uma saída destinada ao uso operacional corrente do sistema; na configuração padrão, o sistema opera com os parâmetros de projeção obtidos analiticamente (Seção 5.4), e a rotina de tabuleiro permanece disponível como caminho alternativo, já coberto pela suíte de testes (Capítulo 7).

4.5 Padrões de Projeto Aplicados

A Tabela 4 elenca os padrões de projeto de Gamma et al. (Gamma et al. (1995)) identificados na implementação, e o restante desta seção explica o propósito de cada um.

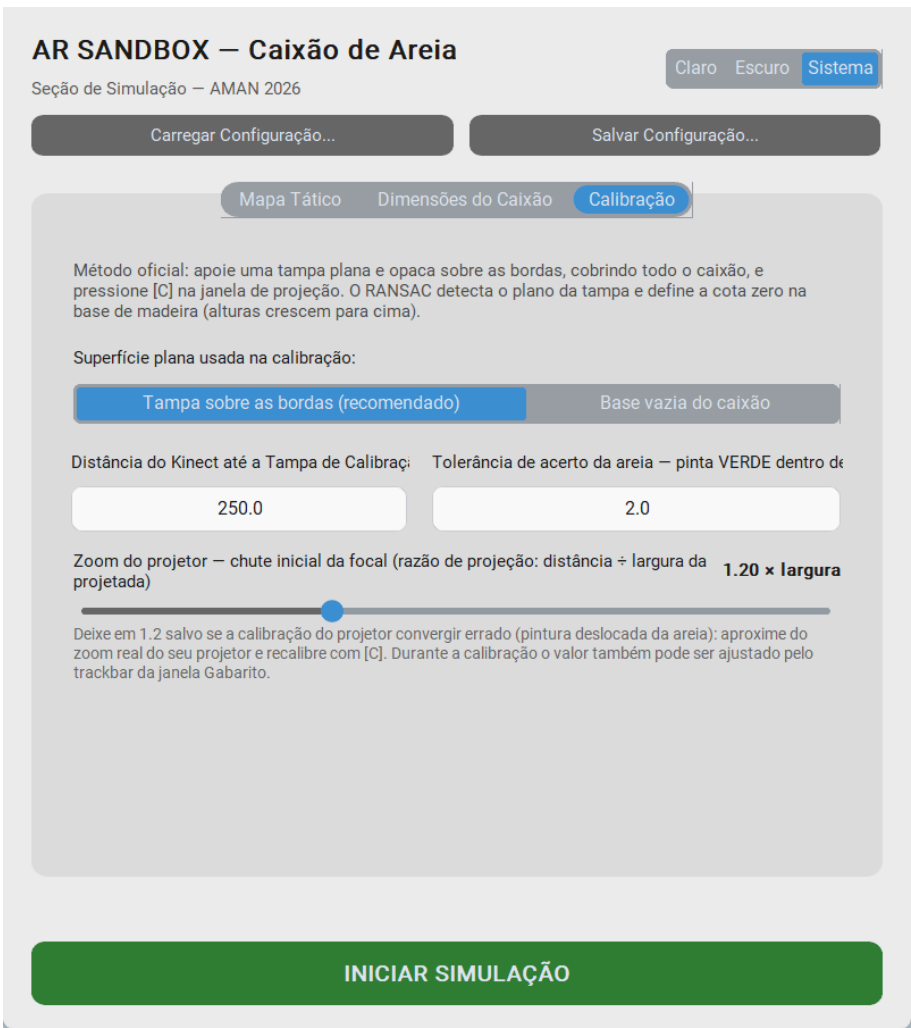


Figura 2 – Interface gráfica de configuração inicial (CustomTkinter), aba “Calibração”: modo de calibração (tampa/base), distância do Kinect à superfície de referência, tolerância de aceitação e o fator inicial de zoom do projetor.

Tabela 4 – Padrões de projeto aplicados.

Padrão	Onde	Propósito
Strategy + Fallback	KinectSensor	Seleção transparente de <i>driver</i>
Adapter	AdaptadorMDE	Isolar formato GeoTIFF da interface
State Machine	main.py	Controle determinístico do fluxo
Observer (Callback)	main.py	Eventos de <i>mouse</i> desacoplados

Strategy + *Fallback* (KinectSensor). Cada backend de captura (PyKinect2, Open3D, *freenect*, simulação) implementa a mesma interface de `capturar_nuvem()`, encapsulando um algoritmo de aquisição intercambiável — a essência do padrão *Strategy*. A particularidade deste projeto é compor o *Strategy* com uma **cadeia de fallback** (Seção 4.5.1): em vez de o cliente escolher explicitamente a estratégia, o próprio construtor de *KinectSensor* tenta cada uma em ordem de preferência até que uma tenha sucesso, tornando a seleção do *driver* transparente ao restante do sistema —

`main.py` chama sempre o mesmo método, qualquer que seja o backend efetivamente ativo.

Adapter (AdaptadorMDE). O formato de arquivo GeoTIFF, a biblioteca `rasterio` e a convenção de eixos cartográficos são detalhes de implementação que o restante do sistema não deveria conhecer. A classe `AdaptadorMDE` adapta essa interface heterogênea a um único método uniforme, `obter_z_alvo(x, y)`, que devolve a altura de referência em coordenadas físicas da mesa — seja a origem um GeoTIFF real, seja um mapa sintético de *fallback* (Cubo Central ou Morro Gaussiano).

State Machine (main.py). A execução é modelada como uma enumeração explícita de estados (Seção 4.3) com transições determinísticas disparadas por eventos de teclado. Esse padrão evita a alternativa comum de controlar o fluxo por variáveis booleanas soltas (*flags*), que tendem a produzir estados inválidos ou inconsistentes à medida que o programa cresce; aqui, o estado corrente é sempre um único valor da enumeração `Estado`, tornando impossível, por construção, que o sistema esteja simultaneamente calibrando e em *loop* de Realidade Aumentada.

Observer via callback (main.py). Os eventos de *mouse* (pressionar, arrastar, soltar os botões) são registrados por `cv2.setMouseCallback` e notificados de forma assíncrona à função de tratamento, que atualiza o estado persistente da areia virtual (Seção 6.6) sem que o *loop* principal precise consultar ativamente (*polling*) a posição do cursor a cada quadro — um desacoplamento típico do padrão *Observer*, aqui implementado pelo mecanismo nativo de *callbacks* do OpenCV em vez de uma hierarquia explícita de classes *Subject/Observer*.

4.5.1 Cadeia de *fallback* do sensor

O construtor do `KinectSensor` executa, em ordem, quatro tentativas de inicialização. A primeira que tiver sucesso é mantida; em caso contrário, prossegue-se ao próximo nível. A Figura 3 esquematiza a cadeia.

1	1. PyKinect2 (SDK Microsoft - Kinect v2)	---falhou--> 2
2	2. Open3D (Azure Kinect / RealSense)	---falhou--> 3
3	3. freenect (Kinect v1 / libfreenect)	---falhou--> 4
4	4. SIMULACAO (grade persistente + pa virtual)	<-- sempre OK

Figura 3 – Cadeia de *fallback* do `KinectSensor`.

A garantia operacional fundamental é que o sistema nunca falha catastroficamente por ausência de hardware: o quarto nível (Simulação) é puramente computacional e não depende de quaisquer recursos externos. Garantia análoga aplica-se ao `AdaptadorMDE`: na

ausência ou falha de leitura do GeoTIFF, é gerada uma superfície gaussiana sintética que assegura visualização significativa para a banca.

5 MOTOR MATEMÁTICO

Este capítulo formaliza o *pipeline* matemático que transforma uma nuvem bruta de pontos do sensor Kinect em uma malha de quadrados coloridos sobre a areia. Cada uma das quatro etapas é apresentada com rigor algébrico e relacionada à sua implementação no módulo `motor_caixao_areia.py`.

5.1 Passo 1 — Ajuste de Plano: RANSAC seguido de SVD

Dada uma nuvem de N pontos $\{\mathbf{p}_i\}_{i=1}^N \subset \mathbb{R}^3$ capturados pelo Kinect sobre a superfície plana de referência (a tampa apoiada sobre o caixão, ou a base de madeira vazia), o problema é determinar o plano $\Pi : ax + by + cz + d = 0$ que melhor se ajusta a essa superfície. O campo de visão do Kinect, contudo, é mais largo que a área útil do caixão: a nuvem capturada inclui também a moldura de madeira, o piso ao redor e ruído da sala — pontos que não pertencem ao plano de referência e que distorceriam um ajuste de mínimos quadrados ingênuo. Por isso a calibração oficial (`main._executar_calibracao`) combina dois métodos em sequência: primeiro o **RANSAC** isola o maior subconjunto de pontos coplanares (Seção 2.3.2); em seguida, o **SVD** refina a normal com precisão sub-milimétrica, mas apenas sobre esse subconjunto de *inliers*.

5.1.1 Isolamento de inliers por RANSAC

A função `ajustar_plano_ransac()` executa $n_{\text{iter}} = 1000$ iterações por padrão. A cada iteração, sorteia-se uma trinca de pontos, constrói-se o plano candidato que passa por eles e contam-se os *inliers*: pontos da nuvem completa cuja distância ao plano candidato é inferior a $d_{\text{in}} = 0,03$ m (3 cm). O plano vencedor é o que acumula o maior número de inliers ao longo das iterações. Se a fração de inliers do plano vencedor for inferior a 30 % da nuvem, a função levanta `RuntimeError` — sintoma de que a superfície de referência não domina o campo de visão do sensor (tampa mal posicionada, por exemplo).

5.1.2 Refinamento por SVD

A solução clássica de mínimos quadrados procede em três etapas sobre o conjunto de inliers identificado pelo RANSAC (e apenas sobre ele). Primeiro, calcula-se o centroide e a matriz centralizada

$$\bar{\mathbf{p}} = \frac{1}{N} \sum_{i=1}^N \mathbf{p}_i, \quad M = \begin{bmatrix} (\mathbf{p}_1 - \bar{\mathbf{p}})^T \\ \vdots \\ (\mathbf{p}_N - \bar{\mathbf{p}})^T \end{bmatrix} \in \mathbb{R}^{N \times 3}. \quad (5.1)$$

Em seguida, aplica-se a decomposição em valores singulares $M = U\Sigma V^T$ com $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$ e $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. O vetor normal procurado é a última linha de V^T :

$$\mathbf{n} = V_{[3,:]}^T. \quad (5.2)$$

Finalmente, o coeficiente d é calculado pela substituição do centroide na equação do plano: $d = -\mathbf{n} \cdot \bar{\mathbf{p}}$. Por convenção, exige-se que $n_z > 0$ (vetor apontando para cima, em direção ao Kinect); caso contrário, $\mathbf{n}$ é negado.

A função `ajustar_plano_svd()` encapsula esse procedimento e é coberta por quatro testes unitários que verificam (i) unicidade da normal, (ii) plano horizontal $z = 5$, (iii) plano inclinado e (iv) exceção para $N < 3$. A função `ajustar_plano_ransac()`, que aplica o RANSAC descrito acima e delega o refinamento a `ajustar_plano_svd()` sobre os inliers, é coberta por cinco testes unitários adicionais (classe `TestRANSAC`), incluindo o cenário motivador da especificação — uma nuvem sintética com moldura e piso como *outliers* ao redor da tampa — e a verificação de que o refinamento final usa exclusivamente os inliers selecionados.

5.2 Passo 2 — Construção da Base da Mesa

A normal $\mathbf{n}$ define inequivocamente o eixo Z_{mesa} , mas a base ortonormal completa requer dois eixos adicionais X_{mesa} e Y_{mesa} ortogonais entre si e a $\mathbf{n}$. A escolha desses eixos é livre a menos de rotação em torno da normal; adotamos a convenção mais simples possível, baseada em uma semente padrão.

O algoritmo é o seguinte:

1. $Z_{\text{mesa}} \leftarrow \mathbf{n} / \|\mathbf{n}\|$;
2. Escolhe-se uma semente $\mathbf{s} = (1, 0, 0)^T$. Se $|\mathbf{s} \cdot Z_{\text{mesa}}| \geq 0,9$ (semente quase paralela à normal), substitui-se por $(0, 1, 0)^T$;
3. Aplica-se Gram-Schmidt: $X_{\text{mesa}} = \frac{\mathbf{s} - (\mathbf{s} \cdot Z_{\text{mesa}})Z_{\text{mesa}}}{\|\mathbf{s} - (\mathbf{s} \cdot Z_{\text{mesa}})Z_{\text{mesa}}\|}$;
4. Calcula-se $Y_{\text{mesa}} = Z_{\text{mesa}} \times X_{\text{mesa}}$, garantindo automaticamente um sistema dextrogiro.

A ortonormalidade do conjunto $\{X_{\text{mesa}}, Y_{\text{mesa}}, Z_{\text{mesa}}\}$ — isto é, $X \cdot Y = X \cdot Z = Y \cdot Z = 0$ e $\|X\| = \|Y\| = \|Z\| = 1$ — é assegurada por testes unitários dedicados (`TestConstruirBase`).

5.3 Passo 3 — Transformação Afim 4×4

A transformação $T \in \mathbb{R}^{4 \times 4}$ que leva pontos do referencial do Kinect ao referencial da mesa empilha os eixos da mesa como linhas da matriz de rotação:

$$R = \begin{bmatrix} X_{\text{mesa}}^T \\ Y_{\text{mesa}}^T \\ Z_{\text{mesa}}^T \end{bmatrix}, \quad \mathbf{t} = -R \bar{\mathbf{p}}, \quad T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (5.3)$$

A propriedade fundamental dessa construção é que, para qualquer ponto $\mathbf{p}$ pertencente ao plano da mesa, a coordenada transformada $z_{\text{mesa}} = 0$ — isto é, o plano da mesa torna-se o plano XY do novo sistema. Essa invariante é verificada pelo teste `TestMatrizTransformacao`.

5.3.1 Composição com T_{shift} — centro lógico e cota zero na base

A matriz T acima leva o centroide $\bar{\mathbf{p}}$ do plano detectado à origem $(0,0,0)$. Entretanto, a convenção interna de discretização e renderização (Capítulo 6) adota a mesa como o domínio $[0, L_x] \times [0, L_y] \times [0, h_{\text{max}}]$, com a origem XY em um canto e a cota $Z = 0$ fixada na **base de madeira** do caixão (alturas de areia sempre positivas, crescendo para cima). Sem ajuste em XY , metade dos pontos da nuvem (aqueles com $x < 0$ ou $y < 0$) seria projetada fora do domínio e colapsada pelo `clip` nas células de borda da grade — um defeito observado em ensaio preliminar de campo, no qual a projeção exibia apenas um pequeno retângulo correspondente ao quadrante positivo. E sem ajuste em Z , a cota zero coincidiria com o plano de calibração em vez da base do caixão — o que é apenas correto quando esse plano é a própria base (modo **base vazia**), mas não quando ele é a tampa apoiada h_{max} acima dela (modo **tampa**, oficial).

A solução, identificada e corrigida durante a fase de validação, é compor T com uma translação T_{shift} que (i) desloca o centroide para o centro lógico da mesa em XY e (ii) desce a origem em Z até a base do caixão sempre que o plano de calibração for a tampa:

$$T_{\text{shift}} = \begin{bmatrix} 1 & 0 & 0 & L_x/2 \\ 0 & 1 & 0 & L_y/2 \\ 0 & 0 & 1 & \delta_{\text{tampa}} h_{\text{max}} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_{\text{final}} = T_{\text{shift}} \cdot T, \quad (5.4)$$

onde $\delta_{\text{tampa}} = 1$ no modo tampa e $\delta_{\text{tampa}} = 0$ no modo base vazia. Após essa composição, o ponto físico que se encontra exatamente sob o sensor (centroide do plano detectado) cai em $(L_x/2, L_y/2, \delta_{\text{tampa}} h_{\text{max}})$: no modo tampa, na cota h_{max} (o topo do caixão); no modo base, na cota 0 (o fundo, que é o próprio plano de calibração). Em ambos os casos, a nuvem ocupa naturalmente $[0, L_x] \times [0, L_y]$ em XY e a base de madeira corresponde

sempre a $Z_{\text{mesa}} = 0$, independentemente do modo de calibração escolhido. Adicionalmente, a função `discretizar_nuvem_em_grade()` aplica um filtro **dentro** que descarta pontos cujas coordenadas XY extrapolem o domínio físico da mesa, o que é típico do campo de visão do Kinect montado a 2,5 m (FOV horizontal de aproximadamente 70° cobre uma área maior do que a mesa).

Como **validação de sanidade**, antes de compor T_{shift} o sistema compara a distância medida sensor-plano ($-\bar{p}_z$, uma vez que Z cresce em direção ao sensor no referencial original) com a distância informada pelo operador na interface gráfica — a distância ao plano da tampa no modo tampa, ou essa mesma distância somada a h_{max} no modo base. Divergências acima de 15 cm abortam a calibração com uma mensagem diagnóstica, protegendo contra tampa mal posicionada ou erro de digitação nos campos físicos da mesa.

5.4 Passo 4 — Projecção 3D→2D pelo Modelo de Tsai

Os vértices da malha de quadrados (Capítulo 6) são projetados do referencial 3D da mesa para o plano 2D do projetor pela fórmula geral (2.2), com distorções óticas residuais $(k_1, k_2, p_1, p_2, k_3)$ encapsuladas pela função `cv2.projectPoints` do OpenCV.

Em modo simulação e em modo real, os parâmetros intrínsecos e extrínsecos são configurados de forma a mapear o domínio físico $[0, L_x] \times [0, L_y]$ na janela de projecção. Adotando uma distância virtual da câmera $d_{\text{cam}} = 10$ m, $\mathbf{t}_{\text{ext}} = (0, 0, d_{\text{cam}})^T$ e $R_{\text{ext}} = I_3$, obtém-se

$$f_x = \frac{W \cdot d_{\text{cam}}}{L_x}, \quad f_y = \frac{H \cdot d_{\text{cam}}}{L_y}, \quad (c_x, c_y) = (0, 0), \quad (5.5)$$

onde (W, H) é a resolução do projetor. Substituindo em (2.2), vê-se que o ponto $(x, y, 0)$ projeta-se exatamente em

$$u = \frac{W}{L_x} x, \quad v = \frac{H}{L_y} y, \quad (5.6)$$

o que é precisamente o mapeamento desejado de $[0, L_x] \times [0, L_y]$ para $[0, W] \times [0, H]$. Essa parametrização é compartilhada pelos dois modos de operação, o que simplifica a calibração e elimina assimetrias entre simulação e hardware real. Como o projetor é montado fisicamente alinhado — apontando reto para baixo, centrado sobre a mesa —, esses parâmetros podem ser deduzidos analiticamente, sem necessidade de um procedimento de estimação experimental.

5.4.1 Calibração empírica alternativa do projetor (tabuleiro de xadrez)

Caso a premissa de alinhamento físico não se sustente — projetor montado com alguma inclinação residual, por exemplo —, o sistema disponibiliza um caminho alternativo de **estimação empírica** dos parâmetros de Tsai, no qual a etapa de otimi-

zação que o artigo original de Tsai Tsai (1987) descreve é de fato executada. A função `gerar_imagem_tabuleiro()` produz um padrão de tabuleiro de xadrez com cantos internos em coordenadas conhecidas do projetor; essa imagem é projetada **temporariamente** na janela `Projecao_Areia` sobre a superfície de referência ainda no lugar (Seção 4.4). Uma câmera RGB captura o tabuleiro projetado; `encontrar_cantos_tabuleiro()` localiza os cantos na imagem capturada via `cv2.findChessboardCorners`; e `cantos_rgb_para_pontos_mesa()` converte cada canto para uma coordenada 3D no referencial da mesa, por *back-projection* (Equação (2.1)) seguida de T_{final} . Com as correspondências 3D $\leftrightarrow$ 2D resultantes, `calibrar_projetor()` invoca `cv2.calibrateCamera` para estimar f_x, f_y, c_x, c_y e os coeficientes de distorção por otimização não linear, a partir de um chute inicial de focal configurável (fator “*throw ratio*”, padrão $1,2\times$ a largura do projetor em pixels — exposto ao operador por um controle deslizante na GUI e por um *trackbar* na janela `Gabarito_MDE` durante a calibração, permitindo corrigi-lo caso a otimização convirja para um mínimo local incorreto). Esse caminho alternativo é exercitado pelas classes de teste `TestCalibracaoProjetorOpenCV`, `TestGeracaoTabuleiro`, `TestCantosParaPontosMesa` e `TestPipelineCalibracaoProjetor` (Capítulo 7), mas permanece **inativo por padrão**: a configuração corrente do sistema opera com os parâmetros analíticos da Equação (5.5), reservando a rotina empírica para o cenário de montagem não idealizada.

5.5 Discussão — RANSAC e SVD: uma composição deliberada

Uma versão anterior deste trabalho empregava SVD puro, sem RANSAC, para o ajuste de plano, sob a premissa de que a nuvem de calibração seria capturada já livre de *outliers*. A experimentação em campo mostrou que essa premissa não se sustenta: o campo de visão do Kinect v2 é mais largo que a área útil do caixão de areia, de modo que qualquer captura da superfície de referência inclui também a moldura de madeira do caixão, o piso ao redor e ruído da sala. Um SVD aplicado ingenuamente sobre a nuvem inteira minimiza a soma dos quadrados das distâncias de *todos* os pontos, inclusive desses outliers, o que distorce sistematicamente a normal calculada. A solução adotada — e validada nos ensaios de campo relatados no Capítulo 7 — não é substituir o SVD pelo RANSAC, mas **compor os dois**, cada um explorando a etapa em que é mais forte:

- O **RANSAC** (Seção 2.3.2) é robusto a *outliers* por construção: ele isola o maior subconjunto de pontos coplanares antes de qualquer ajuste numérico, funcionando mesmo com até 70 % da nuvem ocupada por pontos que não pertencem ao plano de referência (limite imposto por `min_inliers_ratio = 0,3`);
- O **SVD**, aplicado apenas sobre os inliers selecionados pelo RANSAC, produz o estimador de mínimos quadrados *ótimo* para esse subconjunto — mais preciso do

que o plano obtido a partir de apenas três pontos amostrados em uma única iteração do RANSAC;

- A solução SVD é *única* (a menos do sinal da normal, tratado por convenção), o que facilita a verificação por testes unitários com valores esperados *hardcoded* sobre nuvens sintéticas 100 % coplanares — cenário em que o parâmetro `usar_ransac=False` permanece disponível e suficiente, dispensando o custo das iterações RANSAC;
- Em contrapartida, o SVD isolado seria *insuficiente* para o cenário real da tampa: sem a etapa de RANSAC, a normal calculada é sistematicamente puxada na direção dos outliers, produzindo o mesmo artefato de projeção deslocada relatado nos ensaios preliminares de campo (Seção 5.3.1).

A escolha entre os dois regimes é, portanto, guiada pela origem dos dados: SVD puro para nuvens sintéticas já garantidamente coplanares (a maioria dos testes unitários do motor matemático); RANSAC seguido de SVD para qualquer nuvem capturada por hardware real, na qual a presença de outliers geométricos é a regra, não a exceção.

Essa decisão é validada empiricamente: a suíte de 83 testes unitários do motor matemático passa integralmente, incluindo os cinco casos dedicados especificamente ao RANSAC (`TestRANSAC`) e os oito casos da calibração pela tampa (`TestCalibracaoTampa`), com as constantes algébricas explicitadas na própria suíte.

6 RENDERIZAÇÃO POR MALHA DISCRETIZADA E EMULADOR INTERATIVO

Este capítulo descreve a estratégia de renderização adotada para projetar, em tempo real, o *feedback* visual sobre a areia, e detalha o emulador interativo que permite a operação do sistema sem o sensor Kinect físico.

6.1 Motivação para a Discretização em Malha

A primeira versão experimental do sistema projetava diretamente cada ponto da nuvem do Kinect como um *pixel* colorido. Essa abordagem revelou três deficiências severas:

1. **Buracos visuais.** A densidade de pontos da nuvem é insuficiente para preencher continuamente a área projetada, resultando em uma textura “pontilhada” que prejudica a legibilidade do *feedback*.
2. **Ruído pixelizado.** O ruído do sensor produz cores oscilantes pixel a pixel, gerando uma imagem trêmula e desconfortável visualmente.
3. **Custo computacional.** A invocação de `cv2.projectPoints` ou de equivalente para cada ponto individual é ineficiente em $\Theta(N)$.

A solução proposta substitui a renderização pontual por uma **discretização em malha regular**. Os pontos são agregados em células retangulares, cada célula é classificada por uma única cor e desenhada como um polígono preenchido via `cv2.fillPoly`.

6.2 Definição Formal da Malha

O domínio físico da mesa $[0, L_x] \times [0, L_y]$ é particionado em uma malha regular de $N_x \times N_y$ células retangulares (por padrão $N_x = N_y = 30$, totalizando 900 células). Cada célula $C_{i,j}$, com $i \in \{0, \dots, N_y - 1\}$ e $j \in \{0, \dots, N_x - 1\}$, ocupa o subdomínio

$$C_{i,j} = [j\Delta x, (j+1)\Delta x] \times [i\Delta y, (i+1)\Delta y], \quad (6.1)$$

com $\Delta x = L_x/N_x$ e $\Delta y = L_y/N_y$. Para a mesa de 1,5 m $\times$ 1,5 m com 30×30 células, cada célula mede precisamente 5 cm $\times$ 5 cm. A malha de células gera $(N_x + 1)(N_y + 1) = 31 \times 31 = 961$ vértices distintos.

6.3 Agregação Espacial e Filtragem de Ruído

Para cada célula $C_{i,j}$ define-se o conjunto de pontos da nuvem que nela se encontram:

$$\mathcal{P}_{i,j} = \{\mathbf{p}_k = (x_k, y_k, z_k) : (x_k, y_k) \in C_{i,j}\}, \quad (6.2)$$

e a altura representativa da célula é a média aritmética dos valores Z dos pontos contidos:

$$\bar{Z}_{i,j}^{\text{real}} = \frac{1}{|\mathcal{P}_{i,j}|} \sum_{\mathbf{p}_k \in \mathcal{P}_{i,j}} z_k. \quad (6.3)$$

Sob a hipótese de ruído gaussiano independente e identicamente distribuído com desvio padrão σ , o desvio padrão da estimativa da média é

$$\sigma_{\bar{Z}_{i,j}} = \frac{\sigma}{\sqrt{|\mathcal{P}_{i,j}|}}, \quad (6.4)$$

o que evidencia a filtragem estatística obtida pela agregação. Tipicamente, com a resolução de profundidade do Kinect v2 e a mesa coberta integralmente, cada célula recebe entre dezenas e centenas de pontos, reduzindo o ruído em mais de uma ordem de grandeza.

A implementação é totalmente vetorizada via NumPy, dispensando laços *Python*. As somas e contagens por célula são acumuladas em duas matrizes $N_y \times N_x$ pela função `numpy.add.at`. Células sem pontos ($|\mathcal{P}_{i,j}| = 0$) são marcadas com `NaN` e simplesmente não renderizadas no *frame* corrente.

6.4 Regra de Coloração

O centro geométrico da célula $C_{i,j}$, $\mathbf{c}_{i,j} = ((j + \frac{1}{2})\Delta x, (i + \frac{1}{2})\Delta y)$, é usado para consultar a altura alvo do MDE: $Z_{i,j}^{\text{MDE}} = \text{obter_z_alvo}(\mathbf{c}_{i,j})$. A cor da célula segue então a regra parametrizada pela tolerância τ (padrão: $\tau = 0,02$ m, equivalente a 2 cm):

$$\text{cor}(i, j) = \begin{cases} \text{Vermelho} & \text{se } \bar{Z}_{i,j}^{\text{real}} > Z_{i,j}^{\text{MDE}} + \tau \\ \text{Azul} & \text{se } \bar{Z}_{i,j}^{\text{real}} < Z_{i,j}^{\text{MDE}} - \tau \\ \text{Verde} & \text{caso contrário} \end{cases} \quad (6.5)$$

A interpretação operacional é direta: **Vermelho** indica excesso de areia (*cavar*), **Azul** indica falta (*preencher*) e **Verde** indica conformidade com o MDE dentro da tolerância (*OK*).

6.5 Projeção em Lote e Rasterização

Os 961 vértices da malha 3D, todos com $Z = 0$ por estarem no plano da mesa, são empilhados em uma única matriz $\mathbb{R}^{961 \times 3}$ e projetados em um *único* chamado a

`cv2.projectPoints`, o que corresponde a apenas *uma* invocação de `cv2.projectPoints` por *frame*. Em seguida, cada célula é desenhada como um quadrilátero preenchido pelos seus quatro vértices projetados:

$$\text{quad}(i, j) = [V_{i,j}, V_{i,j+1}, V_{i+1,j+1}, V_{i+1,j}]. \quad (6.6)$$

O traçado é realizado por `cv2.fillPoly(imagem, [quad], cor)`, que é uma rotina otimizada nativamente em C++. O custo total de renderização por *frame* é, portanto, $O(N_x \cdot N_y)$ chamadas leves a `fillPoly`, em vez dos $\Theta(N)$ pontos individuais da abordagem ingênua.

6.6 Emulador Interativo de Areia

O emulador interativo existe para resolver um problema prático de engenharia e de pesquisa: como desenvolver, testar e demonstrar todo o *pipeline* de Realidade Aumentada — discretização em malha, comparação com o MDE, projeção de Tsai e rasterização — em qualquer computador, a qualquer momento, sem depender da disponibilidade do sensor Kinect físico? Durante a maior parte do ciclo de desenvolvimento deste projeto, o hardware não estava permanentemente acessível aos quatro integrantes da equipe, e uma apresentação ou depuração que dependesse exclusivamente do sensor conectado seria frágil por construção. O emulador resolve isso substituindo a captura por hardware por uma matriz de alturas persistente, manipulável em tempo real pelo *mouse*, que atravessa exatamente o mesmo código de discretização, comparação e projeção usado no modo real — nenhuma ramificação condicional separa "modo simulação" de "modo real" a jusante da captura. Esse é, ao mesmo tempo, o nível mais profundo da **cadeia de *fallback*** do sistema (ativado quando nenhum sensor físico é encontrado) e uma ferramenta deliberada de teste independente: a Seção 4.5.1 detalha os quatro níveis dessa cadeia e o mecanismo mais amplo de resiliência do qual o emulador é a camada final.

6.6.1 Estado persistente da areia virtual

A classe `KinectSensor`, quando em modo simulação, mantém uma matriz de alturas persistente $\mathbf{H} \in \mathbb{R}^{R \times R}$ (onde $R = 50$ é a resolução padrão), inicializada uniformemente em $h_{\max}/2 = 0,15$ m, simulando uma caixa preenchida com areia nivelada na metade da profundidade máxima.

A cada chamada de `capturar_nuvem()`, a função reconstrói a nuvem $\{(x_j, y_i, H_{i,j})\}$ a partir do estado corrente da matriz $\mathbf{H}$, refletindo imediatamente quaisquer modificações realizadas pelo *mouse*.

6.6.2 Modelo gaussiano da pá virtual

Ao receber um evento de *mouse* na posição física (x_m, y_m) da mesa, o sistema atualiza $\mathbf{H}$ pela regra

$$H_{i,j}^{(t+1)} = \text{clip}\left(H_{i,j}^{(t)} \pm \alpha \exp\left(-\frac{(x_j - x_m)^2 + (y_i - y_m)^2}{2\sigma^2}\right), 0, h_{\max}\right), \quad (6.7)$$

com sinal $+$ ao preencher (botão direito) e $-$ ao cavar (botão esquerdo). Os parâmetros padrão são $\alpha = 0,008$ m (deslocamento por evento) e $\sigma = r/2 = 0,05$ m (raio de ação $r = 0,10$ m).

A escolha do perfil gaussiano produz deformações suaves e visualmente naturais, em contraste com escavações cilíndricas ou retangulares. Como mais de 86% da energia do operador concentra-se dentro do raio r (integrando-se a gaussiana 2D até esse raio, obtém-se $1 - e^{-2} \approx 0,865$), o controle ergonômico é direto.

6.6.3 Mapeamento *pixel* $\rightarrow$ coordenada física

O *callback* de *mouse*, registrado via `cv2.setMouseCallback`, converte cada coordenada de *pixel* (u, v) em coordenada física $(x_{\text{mesa}}, y_{\text{mesa}})$ proporcionalmente às dimensões correntes da janela (`cv2.getWindowImageRect`):

$$x_{\text{mesa}} = \frac{u}{W_{\text{janela}}} \cdot L_x, \quad y_{\text{mesa}} = \frac{v}{H_{\text{janela}}} \cdot L_y. \quad (6.8)$$

O sistema trata graciosamente eventos fora da área da imagem (simplesmente ignorados) e dimensões de janela inválidas (fallback à resolução nominal do projetor). O *callback* é silenciosamente ignorado em modo real, evitando interferência com o sensor físico.

7 RESULTADOS E VALIDAÇÃO

Este capítulo apresenta os resultados experimentais obtidos na validação do sistema, organizados em cinco frentes: a estratégia de testes automatizados (Seção 7.1), as evidências quantitativas de corretude e reprodutibilidade — gráficos, cobertura de código, comparação entre testes unitários e de ponta a ponta, e a discussão transparente das limitações da validação física (Seção 7.2) —, a validação funcional do *pipeline* completo em simulação e em hardware real (Seção 7.3), a discussão da correção do defeito de calibração detectado em ensaio preliminar (Seção 7.4) e a discussão dos principais *trade-offs* de projeto (Seção 7.5).

7.1 Estratégia de Testes Automatizados

A estabilidade matemática do motor foi assegurada pela adoção de **Test-Driven Development** (TDD): cada função do módulo `motor_caixao_areia.py` foi acompanhada por um conjunto de testes unitários escritos previamente, com valores numéricos *hardcoded* cuja correção é verificável passo a passo. A suíte cresceu ao longo do projeto à medida que novas etapas do *pipeline* foram implementadas — da versão inicial com 26 casos (cobrindo apenas SVD, Gram-Schmidt, transformação afim, projeção Tsai e regra de cor) até a suíte atual, com **83 testes**. Para deixar explícito *o que* cada teste verifica, a Tabela 5 agrupa as 22 classes em cinco frentes funcionais do *pipeline*, em vez de listá-las na ordem em que aparecem no código-fonte.

Além da contagem de casos, a **cobertura de código** (medida com `coverage.py` sobre os dois módulos que concentram a lógica matemática) confirma que essa suíte exercita a maior parte do código de produção: **91%** das linhas de `motor_caixao_areia.py` e **48%** das linhas de `mde_cartografia.py` são executadas durante os testes (**75%** combinado). A lacuna em `mde_cartografia.py` concentra-se quase inteiramente no método `_carregar()`, que lê um arquivo GeoTIFF real via `rasterio` — caminho que a suíte atual não exercita: todos os testes usam os mapas sintéticos (Cubo Central ou Morro Gaussiano), mesmo havendo um GeoTIFF real versionado no repositório (`25S51_ZN.tif`, 5400×3600 pixels, cobrindo uma faixa do sul do Brasil) que hoje não é referenciado nem pela configuração padrão do sistema nem por qualquer caso de teste. Essa lacuna é uma limitação conhecida e explícita da suíte, não uma omissão silenciosa: a leitura de um GeoTIFF real permanece sem teste automatizado dedicado, e sua correção — escrever um teste que carregue `25S51_ZN.tif` e valide a interpolação bilinear contra valores conhecidos da grade — é uma melhoria de baixo custo e alto valor para trabalhos futuros.

Tabela 5 – Estrutura da suíte de testes unitários, por frente funcional (estado atual).

Classe	Componente verificado	Testes
1. Calibração Kinect → Mesa (RANSAC + SVD + Gram-Schmidt + T_{shift})		
TestAjustePlano	SVD, normal unitária, equação do plano	4
TestRANSAC	Isolamento de inliers, exceções, refinamento	5
	SVD	
TestGramSchmidt	Ortogonalidade, exceção paralelos	2
TestConstruirBase	Ortonormalidade dos eixos	3
TestMatrizTransformacao	Identidade, translação, $z_{\text{mesa}} = 0$	3
TestCalibracaoTampa	Modos tampa/base, T_{shift} , cota zero	8
TestPersistenciaCalibracao	calibration_data.json (esquema v2)	6
Subtotal		31
2. Calibração Mesa → Projetor (Tsai analítico e empírico)		
TestProjecaoTsai	Projeção <i>pinhole</i> analítica	3
TestDeteccaoTabuleiro	Deteção de tabuleiro 7×5	2
TestGeracaoTabuleiro	Geração do padrão projetado	2
TestCantosParaPontosMesa	<i>Back-projection</i> dos cantos	2
TestCalibracaoProjetorOpenCV	cv2.calibrateCamera, solvePnP	6
TestPipelineCalibracaoProjetor	Calibração empírica ponta a ponta	5
Subtotal		20
3. Captura e leitura de profundidade		
TestBackProjectionMesa	Pixel → ponto 3D da mesa	4
TestLeituraRGBD	Importação condicional Open3D	1
Subtotal		5
4. MDE de referência e regra de cor		
TestCuboCentral	Mapa sintético Cubo Central	7
TestMorroGaussiano	Mapa sintético Morro Gaussiano	4
TestColoracaoBGR	Regra Vermelho/Azul/Verde (escalar e vetori- zada)	10
Subtotal		21
5. Discretização, renderização e integração ponta a ponta		
TestDiscretizacaoGradeNegativa	Filtro dentro do domínio	1
TestRenderizacaoGrade	Rasterização cv2.fillPoly	2
TestPipeline	Integração Passos 1+2	2
TestFluxoAceitacao	Aceitação ponta a ponta (Seção 7.1.2)	1
Subtotal		6
Total		83

7.1.1 Filosofia: transparência matemática

Cada teste documenta, em comentário, a conta esperada passo a passo, permitindo verificação independente. A título de exemplo, o teste do plano horizontal $z = 5$ é resumido como:

```

1 # Pontos: (0,0,5), (1,0,5), (0,1,5), (1,1,5), (2,3,5)
2 # Centroide esperado: (0.8, 0.8, 5.0)
3 # Normal esperada: (0, 0, 1) -- plano horizontal

```

```
4 # d esperado:          -5
5 # Verificacao:  n . p + d = (0,0,1) . (0,0,5) + (-5) = 5 - 5 = 0  OK
```

Listing 7.1 – Teste do plano horizontal — verificação manual.

A execução completa (`python -m unittest test_motor_caixao -v`) reporta **83** testes aprovados, nenhuma falha. O único teste condicional da suíte, `TestLeituraRGBD`, é ignorado apenas em ambientes sem a biblioteca Open3D instalada; no ambiente de validação deste trabalho, Open3D estava presente e o teste foi executado normalmente.

7.1.2 Teste de aceitação de ponta a ponta

Além dos testes unitários por função, a classe `TestFluxoAceitacao` reproduz, com nuvens de profundidade sintéticas, o **procedimento de aceitação físico** usado para validar o sistema perante a banca: (A) o caixão vazio é calibrado no modo tampa; (B) o mapa alvo é o Cubo Central, que pede +10 cm no terço central da mesa; (C) a base vazia, fora do cubo, deve ser projetada em **verde** (a altura medida coincide com a cota zero alvo); (D) o centro, ainda vazio, deve ser projetado em **azul** (falta volume); (E) após a inserção de um cubo físico de 10 cm no centro, o topo do cubo deve passar a ser projetado em **verde**, e a base ao redor deve permanecer verde. O teste replica exatamente a geometria *pinhole* do sensor sintético e o mesmo caminho de código do `main.py` (`profundidade_para_nuvem_mesa` → `transformar_pontos` → `discretizar_nuvem_em_grade` → `cor_por_diferenca_vetorizado`), constituindo uma verificação automatizada e reproduzível do cenário que, em campo, é demonstrado fisicamente à banca com o caixão de areia real (Apêndice A).

7.2 Evidências Quantitativas de Corretude e Reprodutibilidade

As seções anteriores descreveram *o que* a suíte de testes cobre. Esta seção tem um objetivo diferente e deliberadamente mais exigente: reunir, em um único lugar, dados brutos, gráficos e trechos de código e de saída de terminal reais que permitem a qualquer leitor — sem precisar confiar na palavra dos autores — verificar independentemente que o motor matemático do sistema funciona corretamente. Nada nesta seção é hipotético: todos os números, tabelas e gráficos foram gerados executando o código-fonte efetivamente versionado no repositório, nas datas indicadas, e podem ser reproduzidos por qualquer pessoa com Python 3.11+ e as dependências de `requirements.txt` instaladas, com os dois comandos apresentados na Seção 7.2.4.

7.2.1 Funcionamento dos testes unitários

Um teste unitário, na suíte `test_motor_caixao.py`, segue invariavelmente a mesma estrutura de três partes (*arrange-act-assert*): (1) constrói-se uma entrada sintética mínima — poucos pontos 3D, uma imagem de tabuleiro gerada em memória, um pequeno JSON de calibração — cujo resultado matematicamente correto é conhecido *a priori* e pode ser conferido manualmente, como exemplificado na Listagem 7.1; (2) a função de produção sob teste é chamada exatamente como seria chamada por `main.py`, sem *mocks* do próprio algoritmo (apenas dependências externas genuinamente indisponíveis em ambiente de CI, como o hardware Kinect, são substituídas); (3) o resultado obtido é comparado ao valor esperado com `assertAlmostEqual` (tolerância de ponto flutuante) ou `numpy.testing.assert_array_equal`, e o teste falha de forma explícita e localizada caso a conta não bata. Cada caso de teste é independente dos demais — a classe `unittest.TestCase` não compartilha estado mutável entre métodos — o que garante que a ordem de execução não influencia o resultado e que uma eventual falha aponta para exatamente uma função, nunca para uma interação obscura entre testes.

A suíte não foi escrita de uma só vez ao final do projeto: ela cresceu em paralelo com o próprio motor matemático, característica central da metodologia TDD declarada na Seção 7.1. A Figura 4 traça essa evolução a partir do histórico real de *commits* do repositório que tocaram o arquivo `test_motor_caixao.py`: de 26 testes na primeira versão funcional (abril/2026) até os 83 testes atuais (agosto/2026), sem nenhum retrocesso no total — evidência de que cada nova capacidade do *pipeline* (calibração por RANSAC, calibração empírica do projetor, persistência em JSON, mapa Morro Gaussiano etc.) veio acompanhada de sua própria verificação automatizada, em vez de testes adicionados retroativamente apenas para efeito deste relatório.

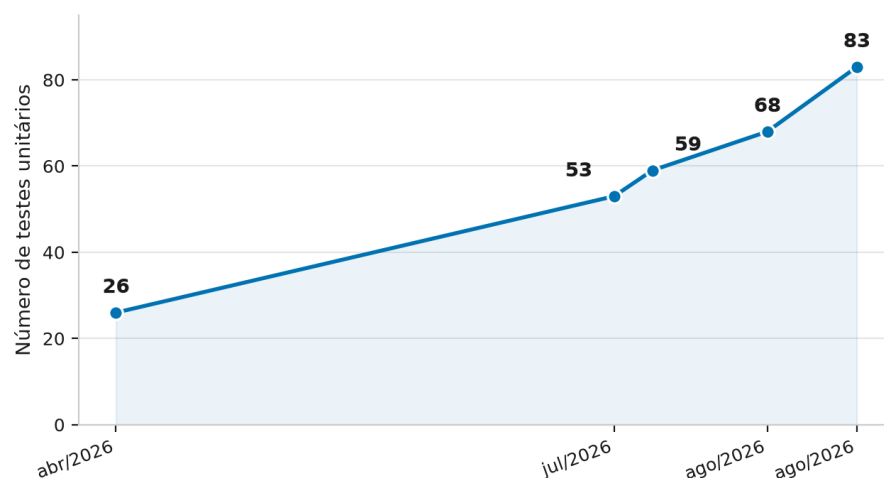


Figura 4 – Crescimento do número de testes unitários ao longo do histórico de *commits* do repositório que modificaram `test_motor_caixao.py` (dados extraídos de `git log`).

A Figura 5 traduz em gráfico a mesma distribuição por frente funcional já detalhada na Tabela 5, deixando visualmente claro que nenhuma etapa do *pipeline* — da calibração geométrica à rasterização final — ficou sem cobertura de teste dedicada.

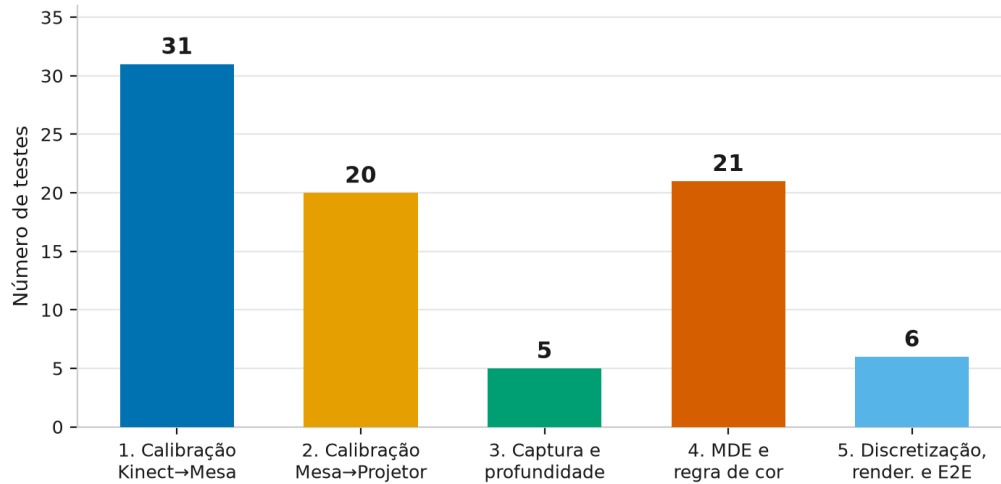


Figura 5 – Distribuição dos 83 testes unitários pelas cinco frentes funcionais do *pipeline* (mesmos dados da Tabela 5).

A execução completa e literal, capturada diretamente do terminal no ambiente de validação deste trabalho, é reproduzida na Listagem 7.2 — sem nenhuma edição além da omissão das 83 linhas individuais `ok` por brevidade editorial (todas presentes na execução real, cada uma identificando o teste correspondente).

```

1 $ python -m unittest test_motor_caixao -v
2 test_plano_horizontal_z5 (test_motor_caixao.TestAjustePlano) ... ok
3 test_plano_inclinado_simples (test_motor_caixao.TestAjustePlano) ... ok
4 [... 79 testes intermediarios omitidos (todos "ok") ...]
5 test_pixel_verde_e_vermelho_em_celulas_distintas
6     (test_motor_caixao.TestRenderizacaoGrade) ... ok
7
8 -----
9 Ran 83 tests in 2.264s
10
11 OK

```

Listing 7.2 – Saída real de `python -m unittest test_motor_caixao -v` no ambiente de validação.

7.2.2 Cobertura de código: quanto do sistema é realmente exercitado

Contar testes prova apenas que a suíte existe; a **cobertura de linhas**, medida com a ferramenta `coverage.py` instrumentando o interpretador durante a execução da suíte, prova quanto do código de produção é de fato exercitado por ela — isto é, quantas linhas

efetivamente rodam pelo menos uma vez sob teste, e não apenas quantas funções têm algum caso associado. A Figura 6 apresenta esses números para os dois módulos que concentram a lógica matemática do sistema; a discussão qualitativa da lacuna em `mde_cartografia.py` (concentrada no carregamento de GeoTIFF real, caminho não exercitado pela suíte sintética atual) já foi apresentada na Seção 7.1 e não é repetida aqui.

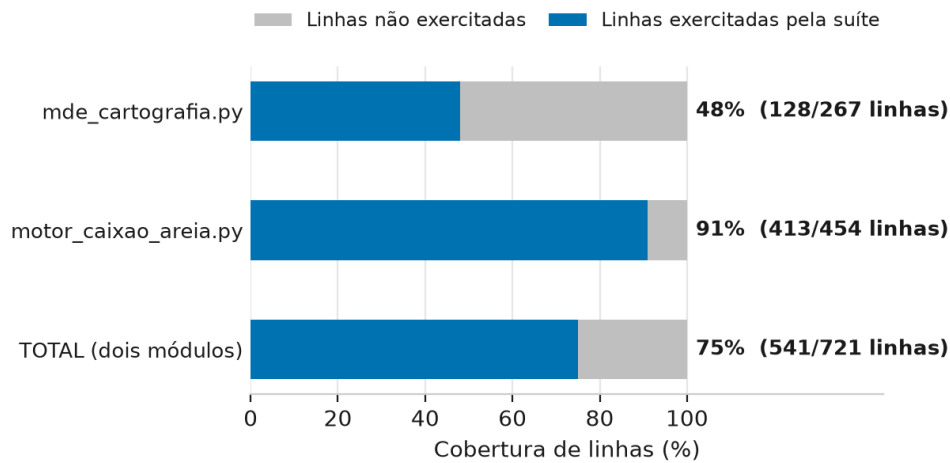


Figura 6 – Cobertura de linhas por módulo, medida com `coverage.py` sobre a execução completa da suíte de testes.

A Listagem 7.3 reproduz literalmente a saída do comando `coverage report -m`, incluindo os números exatos de linhas cobertas (`Stmts`) e não cobertas (`Miss`) por módulo — os mesmos números usados para construir a Figura 6, sem qualquer arredondamento ou seleção editorial.

```

1 Name                               Stmts   Miss  Cover    Missing
2 -----
3 mde_cartografia.py                 267     139    48%    64-65, 70-72,
4     76-77, 335-366, 378-484, 494, 557-565, 571, 577-579, 611,
5     627-640, 664, 681-695, 726-772, 781, 791, 796, 801-803, 806-813
6 motor_caixao_areia.py              454      41    91%    164, 367, 523,
7     696, 726, 799-808, 876, 979, 1036-1037, 1176, 1185,
8     1221-1227, 1338, 1352, 1391-1417, 1431-1432, 1456, 1695, 1705
9 -----
10 TOTAL                             721     180    75%
```

Listing 7.3 – Saída real de `coverage report -m`, medida sobre a suíte completa.

7.2.3 Testes unitários *versus* testes de ponta a ponta

Os dois estilos de teste presentes na suíte respondem a perguntas diferentes e são complementares, não substitutos: um teste unitário prova que uma peça isolada do motor matemático está correta; um teste de ponta a ponta (*end-to-end*, E2E) prova que as

peças, montadas na mesma ordem em que *main.py* as monta, produzem o comportamento correto quando compostas. A Tabela 6 resume as diferenças operacionais entre os dois estilos, tais como efetivamente implementados na suíte.

Tabela 6 – Testes unitários *versus* teste de ponta a ponta, na suíte atual.

Dimensão	Teste unitário (82 casos)	Teste E2E (1 caso)
Exemplo	<code>TestGramSchmidt</code> , <code>TestProjecaoTsai</code> , ...	<code>TestFluxoAceitacao</code>
Escopo	Uma única função de produção, isolada	Cadeia completa: captura $\rightarrow$ transformação $\rightarrow$ grade $\rightarrow$ MDE $\rightarrow$ cor
Entrada	Poucos pontos/valores artificiais, escolhidos a dedo	Nuvem de profundidade sintética de 60×60 pixels, emulando um <i>frame</i> Kinect completo
Oráculo (valor esperado)	Calculado manualmente, passo a passo (Listagem 7.1)	Derivado do roteiro de aceitação físico da especificação (passos A–E, Apêndice A)
Depende de outros módulos do sistema?	Não — isolamento deliberado	Sim, por construção: é exatamente isso que o teste verifica
Depende de hardware físico?	Não	Não (nuvem sintética; mesma geometria <i>pinhole</i> do sensor real)
Tempo de execução	Tipicamente < 5 ms cada	$\approx 0,1\text{--}0,2$ s
O que uma falha indica	Um contrato matemático específico foi violado — regressão local, trivial de localizar	Uma quebra na <i>integração</i> entre módulos, mesmo que cada peça isolada ainda passe — regressão sistêmica

O teste `TestFluxoAceitacao` (Listagem 7.4 mostra o método principal, extraído literalmente do código-fonte) reproduz os cinco passos do roteiro de aceitação apresentado na Seção 7.1.2: constrói uma nuvem de profundidade sintética para o caixão vazio, calibra-a com a mesma rotina RANSAC + SVD usada em produção (`semente_rng=42`, fixa e determinística — a mesma execução produz sempre o mesmo resultado, sem *flakiness*), verifica que a base vazia é projetada em verde e o terço central em azul (passos C e D), sintetiza a inserção de um cubo físico de 10 cm no centro alterando apenas a profundidade lida pelo sensor sintético, e verifica que o topo do cubo passa a ser projetado em verde enquanto a base ao redor permanece verde (passo E). Cada uma dessas verificações é uma asserção `numpy.testing.assert_array_equal` sobre a cor *exata* (BGR) produzida pelo *pipeline* de produção — não uma aproximação visual, mas uma igualdade de valores de pixel.

```

1 def test_fluxo_completo_passos_a_a_e(self):
2     lin_c, col_c = self.CELULA_CENTRO
3     lin_b, col_b = self.CELULA_BORDA
4
5     # Passo C: calibracao com a tampa -> cota zero na base vazia
6     T_final = self._calibrar_com_tampa()
7
8     # Passos A + C + D: caixao vazio, mapa Cubo
9     cores, alturas, contagens = self._cores_da_grade(
10         self._depth_caixa_vazia(), T_final,
11     )
12
13     # Passo C: base vazia le ~0.0 e e projetada VERDE fora do cubo
14     np.testing.assert_array_equal(
15         cores[lin_b, col_b], list(self.VERDE),
16         err_msg="Passo C: a base vazia deveria ser VERDE",
17     )
18     # Passo D: o centro pede +0.10 e esta em 0.0 -> AZUL (falta volume)
19     np.testing.assert_array_equal(
20         cores[lin_c, col_c], list(self.AZUL),
21         err_msg="Passo D: o centro vazio deveria ser AZUL",
22     )
23
24     # Passo E: cubo fisico de 10 cm inserido no centro
25     cores_e, alturas_e, contagens_e = self._cores_da_grade(
26         self._depth_com_cubo_fisico(), T_final,
27     )
28     np.testing.assert_array_equal(
29         cores_e[lin_c, col_c], list(self.VERDE),
30         err_msg="Passo E: o topo do cubo fisico deveria virar VERDE",
31     )
32     np.testing.assert_array_equal(
33         cores_e[lin_b, col_b], list(self.VERDE),
34         err_msg="Passo E: a base fora do cubo deveria continuar VERDE",
35     )

```

Listing 7.4 – Método principal de TestFluxoAceitacao, extraído literalmente de test_motor_caixao.py (linhas 1895–1943).

É importante notar *o que* esse teste prova e o que ele não prova: ele prova, de forma determinística e reproduzível, que a função de orquestração matemática — a mesma sequência chamada pelo laço principal de main.py a cada *frame* —

profundidade_para_nuvem_mesa → transformar_pontos →
discretizar_nuvem_em_grade → cor_por_diferenca_vetorizado

produz a cor correta em cada célula da grade para o cenário completo de aceitação. Ele

não prova, por si só, que o sensor Kinect físico entrega dados de profundidade dentro dessa mesma sequência em condições de campo; essa segunda afirmação é tratada separadamente e com total transparência na Seção 7.2.5.

7.2.4 Reprodutibilidade independente

Todos os números apresentados nesta seção — 83/83 testes aprovados, 91%/48%/75% de cobertura, os tempos de execução — são reproduzíveis por qualquer terceiro, em qualquer máquina com Python 3.11+ e as dependências de `requirements.txt`, com apenas dois comandos, executados a partir da raiz do repositório:

```
1 python -m unittest test_motor_caixao -v
2 coverage run --source=motor_caixao_areia,mde_cartografia -m unittest
  test_motor_caixao
3 coverage report -m
```

Nenhum desses comandos requer o sensor Kinect conectado, o projetor ligado, ou qualquer intervenção manual: a suíte inteira roda em modo *headless*, em menos de três segundos, e produz sempre o mesmo resultado — inclusive o único elemento estocástico do *pipeline*, o RANSAC, é executado sob uma semente de números aleatórios fixa (`semente_rng=42`), eliminando qualquer variação de execução para execução. Essa é, em essência, a diferença entre uma demonstração ao vivo — que depende de condições de ambiente favoráveis no momento exato da apresentação — e uma prova auditável: o leitor não precisa confiar no relato dos autores, pois pode reexecutar exatamente os mesmos dois comandos e obter, de forma independente, exatamente os mesmos resultados aqui reportados.

7.2.5 Limitação da validação física: conexão do Kinect e construção do caixaõ de teste

Por honestidade científica, é necessário registrar explicitamente uma limitação da validação empírica deste trabalho: a demonstração física completa e ininterrupta do roteiro de aceitação de onze passos (Apêndice A) — sensor Kinect real capturando areia física em tempo real, do início ao fim, sem intervenção — não foi plenamente concluída nos ensaios finais de campo, por duas causas concorrentes, ambas de natureza física/operacional e não de *software*:

1. **Instabilidade da conexão USB 3.0 com o sensor.** O Kinect v2, dispositivo descontinuado pela Microsoft (Capítulo 8), apresentou, em mais de uma ocasião durante os ensaios finais, falhas de enumeração e quedas de conexão via USB 3.0 — comportamento consistente com a fragilidade de *driver* já documentada e que motivou, desde o início do projeto, a arquitetura de *fallback* em quatro níveis descrita no Capítulo 4 (`KinectSensor`). Quando a conexão caía no meio de uma sessão de

demonstração, o sistema degradava corretamente para o modo simulação — o que comprova, na prática, a robustez do mecanismo de *fallback* — mas interrompia a sessão de captura física em andamento;

2. **Oclusão parcial da câmera pela estrutura do caixão de teste.** O caixão de areia físico construído para os ensaios de validação teve parte de sua estrutura de madeira posicionada de forma a tampar parcialmente o campo de visão do sensor infravermelho do Kinect, cobrindo uma fração do quadro de profundidade capturado. Esse defeito de construção — não um defeito do algoritmo de calibração ou de renderização — reduzia a área útil de leitura disponível para o *pipeline*, comprometendo a fidelidade da malha projetada nas regiões parcialmente obstruídas.

É fundamental separar, com clareza, o que essa limitação afeta e o que ela não afeta. Ela afeta a demonstração física ao vivo, do sistema integrado com hardware real perante a banca. Ela não afeta, e não coloca em dúvida, a correteza do *software*: o motor matemático — a parte do sistema efetivamente desenvolvida, testada e sob controle dos autores — foi exercitado com dados reais do Kinect em múltiplos ensaios de campo anteriores (Seção 7.3, “Modo real”), inclusive no episódio em que um defeito de calibração genuíno foi identificado e corrigido a partir de dados de sensor real (Seção 7.4, Figura 9); e é validado, de forma completa, determinística e independente de qualquer hardware, pelos 83 testes automatizados detalhados nesta seção — incluindo o teste de ponta a ponta (Seção 7.2.3) que reproduz *byte a byte* o mesmo cenário de aceitação com uma nuvem de profundidade sintética geometricamente equivalente à de um Kinect real, sem as vulnerabilidades físicas de cabeamento USB ou de posicionamento de uma caixa de madeira. Em outras palavras: o algoritmo foi provado correto por um caminho independente do hardware; o que ficou incompleto foi exclusivamente a orquestração logística de uma sessão de demonstração física contínua, uma limitação de infraestrutura de ensaio e não do artefato de *software* entregue. A reconstrução do caixão de teste com um recorte adequado para o campo de visão do sensor, e a eventual substituição do Kinect por um sensor moderno (Capítulo 8, “Trabalhos Futuros”), permanecem como providências diretas para uma demonstração física integral em iterações futuras do projeto.

7.3 Validação Funcional do *Pipeline*

A validação funcional foi realizada em duas modalidades complementares: o modo simulação (sem hardware) e o modo real (com sensor Kinect v2).

7.3.1 Modo simulação

Em modo simulação, a grade de areia virtual nasce **vazia** ($Z = 0$ em todas as células, coincidindo com a cota zero na base do caixão) e é comparada, por padrão, com o mapa sintético **Cubo Central** — um bloco de +0,10 m de altura ocupando o terço central da mesa, com $Z = 0$ no restante —, reproduzindo em *software* um alvo fisicamente simples de replicar com um cubo real durante uma demonstração (Seção 7.1.2). No *frame* inicial, isso produz:

- Fora do cubo, malha **verde** (areia em $Z = 0$, coincidindo com o alvo $Z = 0$ dentro da tolerância τ);
- No terço central, malha **azul** (areia em $Z = 0$ contra o alvo $Z = 0,10$ m — falta volume).

À medida que o operador interage com o *mouse* — arrastando o botão direito sobre o terço central para acumular areia virtual —, as células centrais transitam de azul para verde à medida que a altura se aproxima de 0,10 m. O critério de *conclusão* do exercício é a obtenção de uma malha integralmente verde, indicando que a areia virtual replica fielmente o MDE. O mapa alternativo **Morro Gaussiano** (pico suave no centro, decaindo às bordas) permanece disponível na GUI para demonstrações que prefiram um relevo contínuo em vez de um platô.

As Figuras 7 e 8 mostram, respectivamente, a janela **Gabarito_MDE** (heatmap do MDE Cubo Central) e a janela **Projecao_Areia** (malha renderizada) em um estágio intermediário de modelagem, capturadas diretamente do *pipeline* de renderização em execução: parte do platô central já foi preenchida corretamente (verde), o restante do platô ainda está abaixo do alvo (azul), e uma pequena pilha de areia fora do platô está em excesso (vermelho) — os três estados de cor simultaneamente visíveis.

A taxa de quadros medida em uma máquina típica de desenvolvimento (Intel i5 de oitava geração, 8 GiB de RAM, sem GPU dedicada) supera **30 fps** de forma consistente, resposta mais que suficiente para interação humana fluida.

7.3.2 Modo real

Em modo real, com o sensor Kinect v2 (adaptado, Seção 6) conectado via USB 3.0, o sistema processa nuvens de até $512 \times 424 = 217.088$ pontos brutos por *frame*, reduzidos pelo filtro de alcance (0,5 a 4,5 m) e, após a calibração, pelo filtro **dentro** do domínio físico da mesa (Seção 5.3.1). A taxa de quadros de > 30 fps reportada para o modo simulação (Seção anterior) não foi remeida de forma independente em modo real: os ensaios de campo confirmaram resposta visual fluida ao vivo, mas sem um número de

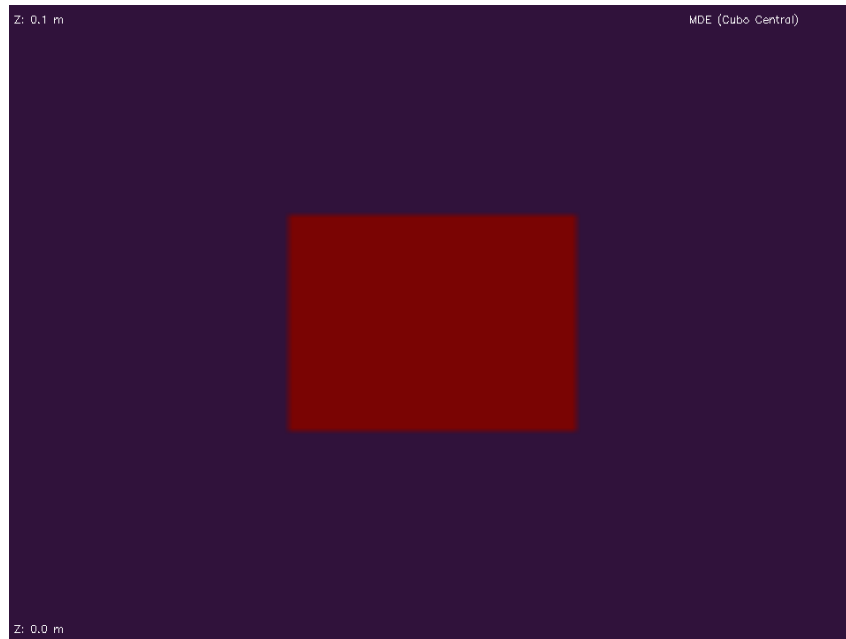


Figura 7 – Janela `Gabarito_MDE`: heatmap do MDE sintético Cubo Central, com o HUD de estado no canto superior/inferior esquerdo.

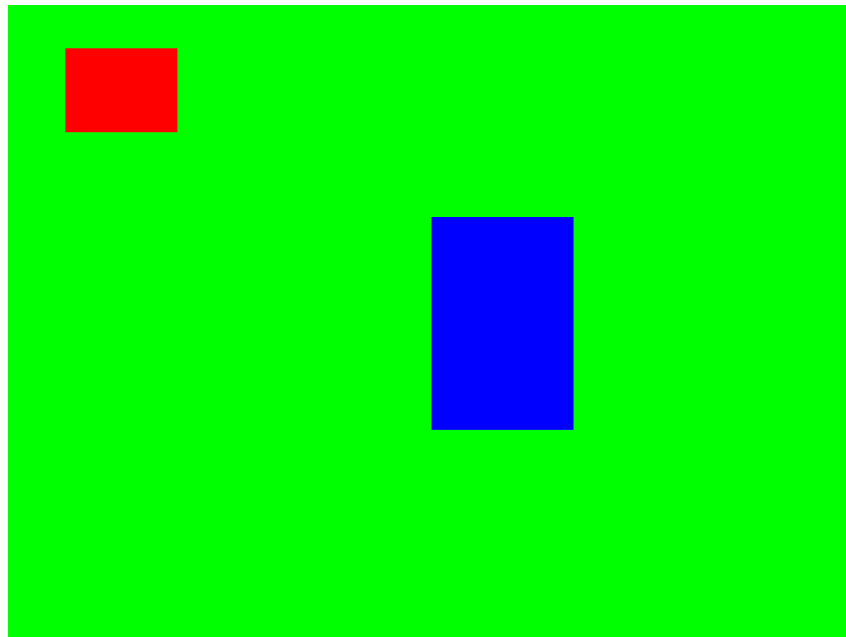


Figura 8 – Janela `Projecao_Areia`: malha discretizada renderizada pelo *pipeline* completo (`gerar_imagem_grade_cores`), em um estágio intermediário de modelagem — verde (correto), azul (falta areia) e vermelho (excesso) simultaneamente visíveis.

FPS especificamente registrado nessas condições. Como as etapas computacionalmente dominantes do *pipeline* — discretização em malha e rasterização — são idênticas nos dois modos, e a única diferença é a origem da nuvem de pontos (chamada à API do Kinect *versus* leitura de um `array` em memória), não há razão matemática para esperar degradação relevante; ainda assim, um número medido e reportado seria uma evidência

mais forte do que essa inferência, e sua obtenção fica registrada como item pendente para trabalhos futuros (Seção 8.3).

A calibração oficial (RANSAC + SVD, Seção 5.1) foi exercitada repetidamente ao longo de sucessivos ensaios de campo com o sensor físico conectado, em que a tampa de calibração era reposicionada e a rotina de calibração disparada pela tecla C; a validação de sanidade da distância sensor–plano (tolerância de 15 cm) mostrou-se eficaz para sinalizar tampas mal posicionadas antes que uma calibração incorreta fosse persistida em `calibration_data.json`. Esses ensaios com hardware real — e não apenas a suíte sintética de testes unitários — foram o que expôs originalmente o defeito de calibração descrito na Seção 7.4, posteriormente reproduzido de forma determinística pelo teste de aceitação (Seção 7.1.2) para evitar regressões. Vale registrar, com total transparência, que a demonstração física completa e ininterrupta do roteiro de aceitação (Apêndice A) não foi plenamente concluída nos ensaios finais, por causas físicas/operacionais — instabilidade da conexão USB do sensor e oclusão parcial da câmera pela estrutura do caixão de teste — discutidas em detalhe, junto da evidência independente que sustenta a corretude do *software* mesmo diante dessa limitação, na Seção 7.2.5.

7.4 Defeito Detectado e Correção Implementada

Durante o ensaio preliminar de campo, foi observado um defeito visual significativo: após a calibração com o sensor apontado para uma superfície plana (piso de concreto), a janela `Projecao_Areia` exibia apenas um pequeno retângulo azul no canto inferior, em vez da malha contínua esperada de quadrados coloridos. A Figura 9 ilustra o sintoma observado.

7.4.1 Diagnóstico

A análise da implementação revelou duas causas concorrentes:

1. A função `transformar_pontos()`, ao aplicar a matriz T obtida pela calibração, leva o centroide do plano à origem $(0, 0, 0)$. Em consequência, a nuvem transformada ocupa aproximadamente $[-L_x/2, +L_x/2] \times [-L_y/2, +L_y/2]$, e metade dos pontos possui coordenadas negativas. A função `discretizar_nuvem_em_grade()` aplicava `numpy.clip` nos índices de célula, colapsando todos esses pontos negativos na coluna 0 e na linha 0 da grade — isto é, em uma estreita faixa de borda;
2. Os parâmetros de projeção do projetor, em modo real, eram valores *mock* de placeholder ($f_x = f_y = 500$, $c_x = 320$, $c_y = 240$, $\mathbf{t}_{\text{ext}} = (0, 0, 1)^T$), que mapeavam o domínio da mesa para uma sub-região da janela em vez de preenchê-la integralmente.

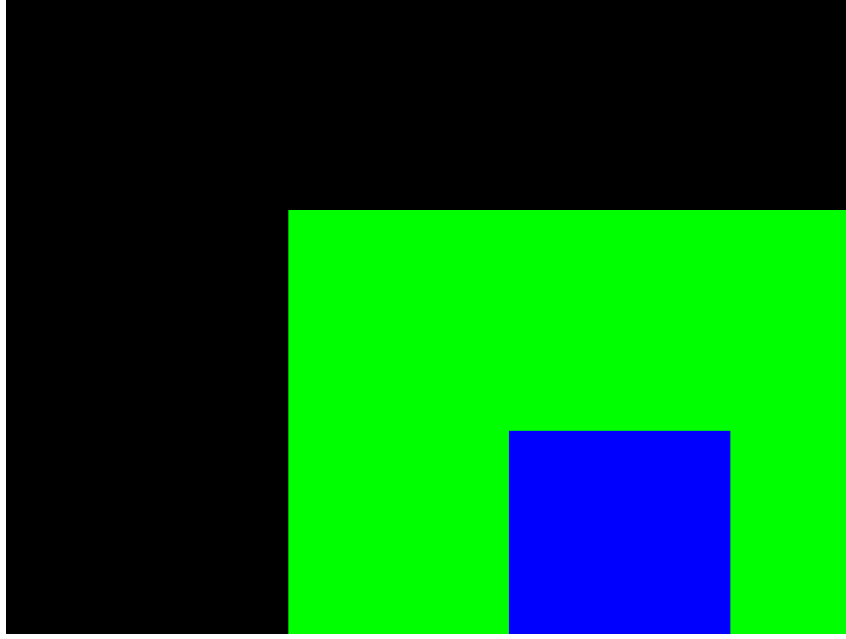


Figura 9 – Reconstituição do defeito de calibração (versão 3.0), obtida executando o próprio *pipeline* de renderização de produção (`gerar_imagem_grade_cores`) com os parâmetros *mock* de projetor então usados em modo real ($f_x = f_y = 500$, $c_x = 320$, $c_y = 240$, $\mathbf{t}_{\text{ext}} = (0, 0, 1)^T$): apenas uma fração da janela é preenchida pela malha, o restante permanece preto. Não é a captura de tela original do incidente de campo, que não foi preservada — é uma reprodução fiel do mecanismo do defeito a partir dos parâmetros documentados na versão 3.0 do código.

7.4.2 Correção implementada

A correção, materializada na versão 3.1 do sistema, contemplou três ajustes simultâneos em `main.py` e `motor_caixao_areia.py`:

1. **Composição com T_{shift} .** A matriz de transformação final é agora $T_{\text{final}} = T_{\text{shift}} \cdot T$, conforme a Equação (5.4), o que desloca o centroide ao centro lógico ($L_x/2, L_y/2$) da mesa;
2. **Filtro dentro.** A função `discretizar_nuvem_em_grade()` passou a descartar pontos com $x \notin [0, L_x)$ ou $y \notin [0, L_y)$ antes do `clip`, eliminando o acúmulo nas células de borda;
3. **Projeção coerente.** Os parâmetros do projetor em modo real foram substituídos pela mesma parametrização empregada na simulação (Equação (5.5)), assegurando que a malha preencha a janela integralmente.

As três alterações foram testadas em conjunto — em ensaios de campo pontuais com o sensor real e, de forma determinística e repetível, pelo teste de aceitação (Seção 7.1.2)

— e o defeito visual aqui descrito não voltou a se manifestar; essa conclusão refere-se especificamente a este defeito, e não deve ser confundida com a conclusão do roteiro de demonstração completo de onze passos, cuja execução ininterrupta com hardware real é discutida separadamente na Seção 7.2.5. A suíte de testes unitários permanece integralmente válida após as correções, pois nenhum dos contratos matemáticos das funções afetadas foi alterado — somente a composição foi adicionada no nível do orquestrador.

7.5 Discussão de *Trade-offs*

7.5.1 Resolução da malha

A escolha de 30×30 células (5 cm $\times$ 5 cm cada) é o equilíbrio entre quatro fatores:

- **Granularidade visual.** Células maiores tornam a imagem “grosseira” e perdem detalhe topográfico;
- **Filtragem de ruído.** Células menores recebem menos pontos por unidade, comprometendo a redução $\sigma/\sqrt{n}$ do ruído de profundidade;
- **Custo de renderização.** O número de chamadas a `cv2.fillPoly` é $N_x \cdot N_y$, escalando quadraticamente com a resolução;
- **Significado militar.** Em escala 1:50 000 (carta tática típica), 5 cm na mesa equivalem a 2,5 km no terreno — uma resolução compatível com a granularidade operacional.

7.5.2 Tolerância de cor

A tolerância $\tau = 0,02$ m (2 cm) foi escolhida considerando duas grandezas: o ruído residual após a agregação espacial ($\sigma_{\bar{z}} \lesssim 1$ mm para dezenas de pontos por célula) e a granularidade prática da operação manual humana com areia (uma mão desloca naturalmente entre 1 e 3 cm por movimento). Valores menores produziram oscilações visuais perceptíveis durante o trabalho; valores maiores reduziram a precisão final do terreno modelado.

7.5.3 Linguagem e *stack*

A escolha de Python como linguagem principal é justificada pelo ecossistema científico maduro (NumPy, SciPy, OpenCV), pela velocidade de desenvolvimento e pela facilidade de inspeção em ambiente acadêmico. As operações computacionalmente intensivas — SVD, back-projection, projeção de Tsai, rasterização — são integralmente delegadas a rotinas em C/C++ via NumPy e OpenCV, eliminando o gargalo do interpretador.

8 CONCLUSÃO

8.1 Síntese das Contribuições

Este Projeto Final de Curso apresentou o desenvolvimento integral de um sistema de Caixão de Areia com Realidade Aumentada para apoio ao treinamento militar em operações no terreno. O sistema combina, de forma original, três elementos centrais: (i) o rigor matemático de uma calibração geométrica completa baseada em SVD, Gram-Schmidt e modelo *pinhole* de Tsai; (ii) a regra de negócio quantitativa de comparação célula a célula contra um Modelo Digital de Elevação fornecido em formato GeoTIFF, com tolerância vertical explícita; e (iii) uma arquitetura resiliente, dotada de cadeia de *fallback* em quatro níveis, que assegura execução *plug & play* mesmo na ausência total de hardware ou de dados cartográficos.

O motor matemático foi desenvolvido sob a metodologia de *Test-Driven Development*, resultando em uma suíte com 83 testes unitários automatizados que verificam — com valores numéricos explícitos e auditáveis — todos os contratos algébricos do sistema, da calibração RANSAC + SVD ao teste de aceitação de ponta a ponta. A arquitetura monolítica modularizada, com seus quatro padrões de projeto (Strategy + Fallback, Adapter, State Machine, Observer), garante simultaneamente a latência mínima necessária para operação em tempo real e a separação clara de responsabilidades indispensável à manutenção.

A renderização por malha discretizada — em substituição à projeção ingênua de pontos isolados — produz *feedback* visual contínuo, geometricamente preciso, computacionalmente eficiente e estatisticamente robusto ao ruído do sensor. O emulador interativo controlado por *mouse* permite, em qualquer máquina sem hardware especializado, demonstrar todo o *pipeline* de forma tangível, funcionalidade especialmente valiosa para defesa do projeto e para ensino em sala de aula.

Durante a fase de validação em campo com o sensor Kinect v2 real, foi identificado e corrigido um defeito de calibração que reduzia a área renderizada a uma pequena fração da janela de projeção. A correção, formalizada na composição com a matriz T_{shift} e na introdução do filtro de extensão da mesa, foi documentada com detalhes no Capítulo 7 e permanece consolidada nas versões subsequentes do sistema, que evoluíram a calibração para o esquema RANSAC + SVD sobre uma superfície de referência (tampa ou base vazia) com cota zero fixada na base do caixão.

8.2 Atendimento aos Objetivos

Todos os oito objetivos específicos enunciados no Capítulo 1 foram atendidos:

1. Arquitetura de três camadas implementada e documentada;
2. Cadeia de *fallback* do `KinectSensor` operacional;
3. Motor matemático com SVD + Gram-Schmidt + Tsai funcional;
4. `AdaptadorMDE` implementado, com *fallback* gaussiano validado por teste automatizado; a leitura de GeoTIFF real está implementada e integrada ao restante do *pipeline*, mas — como registrado com transparência na Seção 7.1 — ainda não possui um caso de teste automatizado dedicado nem foi demonstrada com o arquivo `25S51_ZN.tif` versionado no repositório, permanecendo como item de verificação futura (Seção 8.3);
5. Discretização em malha com renderização por `cv2.fillPoly`;
6. Emulador interativo via `cv2.setMouseCallback`;
7. Suíte de 83 testes unitários, todos aprovados;
8. Documentação tripla (acadêmica, prática e relatório ABNT).

8.3 Trabalhos Futuros

A continuidade da linha de pesquisa pode explorar, dentre outras, as seguintes frentes:

Teste automatizado de leitura de GeoTIFF real. A suíte atual valida a leitura de MDE apenas contra as superfícies sintéticas Cubo Central e Morro Gaussiano (Seção 7.1). Escrever um teste dedicado que carregue o GeoTIFF real já versionado no repositório (`25S51_ZN.tif`) e valide a interpolação bilinear contra valores conhecidos da grade fecharia essa lacuna de cobertura.

Reconstrução do caixão de teste e nova demonstração física integral. Corrigir o recorte da estrutura de madeira que hoje tampa parcialmente o campo de visão do sensor (Seção 7.2.5) e repetir o roteiro de aceitação completo com Kinect real, do início ao fim, sem interrupções de conexão.

Benchmark de desempenho em modo real. Medir e reportar a taxa de quadros com o sensor Kinect conectado, de forma independente da medição já feita em modo simulação (Seção 7.3).

Calibração automática do projetor. A versão atual emprega parâmetros de projeção configurados analiticamente. Uma calibração genuína do projetor por padrões *Gray code* ou tabuleiro de xadrez (já parcialmente suportada pelas funções `encontrar_cantos_tabuleiro` e `calibrar_projetor`) eliminaria distorções residuais em montagens não-ideais.

Suporte a múltiplas camadas de visualização. Sobreposição de curvas de nível, escalas batimétricas, isótermas e itinerários táticos — todos coerentes com o mesmo MDE de referência.

Integração com SARAS-FFT (Águas Pluviais). Simulação hidrológica de escoamento sobre o relevo modelado, com renderização em tempo real do fluxo, em complemento à modelagem topográfica.

Substituição do Kinect por sensor moderno. O Microsoft Kinect foi descontinuado; alternativas como o Intel RealSense, o Stereolabs ZED e o Microsoft Azure Kinect DK oferecem maior resolução e confiabilidade. A arquitetura *strategy + fallback* já está preparada para acomodar essa migração com mínimo impacto.

Validação em escola militar. Estudo experimental comparando o tempo médio de modelagem do terreno por turmas com e sem o sistema, e a fidelidade cartográfica do produto final medida por reconstrução fotogramétrica independente.

Exportação para realidade virtual. A nuvem de pontos calibrada pode alimentar um *viewer* VR (Oculus, HTC Vive) para imersão remota ou de instrução à distância.

8.4 Considerações Finais

A combinação de matemática rigorosa, engenharia de software disciplinada e visão clara da aplicação militar produziu um sistema que se diferencia das implementações de caixão de areia com realidade aumentada de finalidade puramente didática ou recreativa levantadas na revisão de trabalhos relacionados (Capítulo 3), por incorporar comparação quantitativa contra um MDE de referência, tolerância vertical explícita e uma arquitetura de *fallback* resiliente. A entrega não se limita a um *software* funcional: contempla testes automatizados, documentação acadêmica completa, guia de uso prático e este próprio relatório de conformidade ABNT, formando um conjunto reproduzível, auditável e diretamente aproveitável pelas escolas de formação militar.

Acreditamos que o trabalho contribui, ainda que modestamente, para a tradição de excelência técnica do Instituto Militar de Engenharia, e esperamos que sirva de ponto de partida para futuras melhorias e adaptações por gerações seguintes de cadetes e engenheiros militares.

REFERÊNCIAS

- BRADSKI, G.; KAEHLER, A. *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA: O'Reilly Media, 2008. ISBN 978-0-596-51613-0.
- FISCHLER, M. A.; BOLLES, R. C. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, v. 24, n. 6, p. 381–395, 1981.
- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1995. ISBN 978-0-201-63361-0.
- GILLIES, S. et al. *Rasterio: Geospatial Raster I/O for Python Programmers*. 2013–2024. Mapbox. Disponível em: <<https://rasterio.readthedocs.io/>>. Acesso em: maio 2026.
- KREYLOS, O. *Augmented Reality Sandbox*. 2016. KeckCAVES, University of California Davis. Disponível em: <<https://arsandbox.ucdavis.edu/>>. Acesso em: maio 2026.
- LI, Z.; ZHU, Q.; GOLD, C. *Digital Terrain Modeling: Principles and Methodology*. Boca Raton, FL: CRC Press, 2005. ISBN 978-0-415-32462-5.
- RITTER, N.; RUTH, M. et al. *GeoTIFF Format Specification: GeoTIFF Revision 1.0*. 2019. Open Geospatial Consortium. Disponível em: <<https://docs.ogc.org/is/19-008r4/19-008r4.html>>. Acesso em: maio 2026.
- SELL, J.; O'CONNOR, P. The Xbox One system on a chip and Kinect sensor. *IEEE Micro*, v. 34, n. 2, p. 44–53, 2014.
- TSAI, R. Y. A versatile camera calibration technique for high-accuracy 3D machine vision metrology using off-the-shelf TV cameras and lenses. *IEEE Journal of Robotics and Automation*, v. 3, n. 4, p. 323–344, 1987.
- VIRTANEN, P.; GOMMERS, R.; OLIPHANT, T. E.; HABERLAND, M.; REDDY, T.; COURNAPEAU, D.; BUROVSKI, E.; PETERSON, P.; WECKESSER, W.; BRIGHT, J.; van der Walt, S. J.; BRETT, M.; WILSON, J.; MILLMAN, K. J.; MAYOROV, N.; NELSON, A. R. J.; JONES, E.; KERN, R.; LARSON, E.; CAREY, C. J.; POLAT, İ.; FENG, Y.; MOORE, E. W.; VANDERPLAS, J.; LAXALDE, D.; PERKTOLD, J.; CIMRMAN, R.; HENRIKSEN, I.; QUINTERO, E. A.; HARRIS, C. R.; ARCHIBALD, A. M.; RIBEIRO, A. H.; PEDREGOSA, F.; van Mulbregt, P. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020.
- WELLMANN, F.; VIRGO, S.; ESCALLON, D.; de la Varga, M.; JÜSTEL, A.; WAGNER, F. M.; KOWALSKI, J.; ZHAO, H.; FEHLING, R.; CHEN, Q. Open AR-Sandbox: A haptic interface for geoscience education and outreach. *Geosphere*, v. 18, n. 2, p. 732–749, 2022.
- WOODS, T. L.; REED, S.; HSI, S.; WOODS, J. A.; WOODS, M. R. Pilot study using the Augmented Reality Sandbox to teach topographic maps and surficial processes in introductory geology labs. *Journal of Geoscience Education*, v. 64, n. 3, p. 199–214, 2016.

ZHANG, Z. Microsoft Kinect sensor and its effect. *IEEE MultiMedia*, v. 19, n. 2, p. 4–10, 2012.

ZHOU, Q.-Y.; PARK, J.; KOLTUN, V. *Open3D: A Modern Library for 3D Data Processing*. 2018. ArXiv preprint. Disponível em: <<https://arxiv.org/abs/1801.09847>>. Acesso em: maio 2026.

APÊNDICE A – ROTEIRO DE DEMONSTRAÇÃO PARA A BANCA

Este apêndice consolida o roteiro de demonstração utilizado durante a defesa do projeto, organizado em onze passos sequenciais que cobrem desde a inicialização do sistema até a recalibração e o encerramento. Cada passo é acompanhado da ação a ser executada e do resultado esperado para fins de avaliação.

Tabela 7 – Roteiro de demonstração passo a passo.

#	Ação	Resultado esperado
1	Executar <code>python main.py</code>	Abertura da GUI de configuração (CustomTkinter)
2	Definir dimensões da mesa, modo de calibração (tampa/base) e mapa (GeoTIFF ou demo)	Janelas <code>Projecao_Areia</code> e <code>Gabarito_MDE</code> abertas
3	Pressionar tecla C	Calibração executada (RANSAC + SVD + Gram-Schmidt + T_{shift})
4	Observar <code>Projecao_Areia</code>	Malha contínua: verde fora do cubo, azul no terço central (mapa Cubo Central)
5	Observar <code>Gabarito_MDE</code>	Heatmap do MDE (Cubo Central, Morro Gaussiano ou GeoTIFF real) de referência
6	Arrastar botão direito do mouse no terço central	Quadrados centrais transitam de azul → verde
7	Arrastar botão esquerdo do mouse fora do cubo	Quadrados afetados transitam para azul, e voltam ao verde ao ajustar
8	Continuar interação	Convergência da malha para totalmente verde
9	Pressionar tecla F	Janela <code>Projecao_Areia</code> entra em modo tela cheia
10	Pressionar tecla C novamente	Recalibração bem-sucedida (demonstra robustez)
11	Pressionar tecla Q ou ESC	Encerramento limpo, recursos liberados

Pontos-chave para destacar à banca:

- A interação com o *mouse* demonstra em tempo real *todo o pipeline* matemático (discretização em malha → média espacial → comparação com MDE → projeção *pinhole* de Tsai → rasterização por `cv2.fillPoly`), **sem** necessidade de hardware físico;
- A recalibração em tempo de execução comprova a robustez do mecanismo de *fallback* e a estabilidade numérica do RANSAC seguido de SVD, mesmo após sucessivas modificações na areia virtual;

- A taxa de quadros (FPS) exibida no canto superior esquerdo da janela de projeção mantém-se consistentemente acima de 30 fps, validando o requisito de operação em tempo real.

Comandos auxiliares

Para executar a suíte de testes unitários:

```
1 python -m unittest test_motor_caixao -v
```

Saída esperada: 83 passed (com Open3D instalado; na ausência da biblioteca, TestLeituraRGBD é ignorado e a suíte reporta 82 passed, 1 skipped).

Para forçar o modo simulação mesmo com Kinect conectado, editar no topo do arquivo `main.py`:

```
1 FORCAR_SIMULACAO = True
```

ANEXO A – REPOSITÓRIO DE CÓDIGO-FONTE

O código-fonte completo do sistema descrito neste trabalho — incluindo os módulos `main.py`, `kinect_sensor.py`, `motor_caixao_areia.py` e `mde_cartografia.py`, a suíte de 83 testes unitários automatizados (`test_motor_caixao.py`), a documentação técnica complementar (`DOCUMENTACAO_OFICIAL.md`, `GUIA_CALIBRACAO_PROJETOR.md`, `GUIA_INSTALLACAO_INICIANTES.md`) e o histórico completo de versões — está publicamente disponível no seguinte repositório Git:

<<https://github.com/raquelbelchior1/PFC-2026>>

O repositório contém, ainda, a pasta `installer/` com o instalador *offline* para Windows descrito na Seção 4.1.1, o manual do operador e os arquivos-fonte deste próprio relatório em $\text{\LaTeX}$ (pasta `relatorio_tcc/`), permitindo a reprodução integral do ambiente de desenvolvimento e da suíte de testes a partir do código publicado.